01_knn_demo_repo

April 21, 2026

1 k-Nearest Neighbor on CIFAR-10

This notebook demonstrates a from-scratch implementation of the **k-nearest neighbor (k-NN)** classifier on a subset of CIFAR-10.

1.1 Goals

- load and inspect CIFAR-10
- represent images as vectors in a high-dimensional space
- train and evaluate a k-NN classifier
- compare three distance-computation implementations
- select the hyperparameter `k` by cross-validation

1.2 Related source files

- `src/models/knn.py`
- `src/utils/data.py`

1.3 0. What is k-NN?

Given a test point x, the k-NN classifier: 1. computes the distance from x to every training point, 2. finds the k nearest training examples, 3. predicts the most common label among those neighbors.

In this notebook, each image is represented as a vector in $\mathbf{R}^{3072}$, since CIFAR-10 images have shape $32 \times 32 \times 3$.

1.4 1. Setup

The notebook is meant to live in `demos/`, with reusable code in `src/`. The small path adjustment below lets the notebook import from the repository root when it is run locally.

[1]:
```
import sys
from pathlib import Path

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

import random
import numpy as np
```

```
import matplotlib.pyplot as plt

from src.models.knn import KNearestNeighbor
from src.utils.data import load_CIFAR10

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Optional convenience for local development
%load_ext autoreload
%autoreload 2
```

1.5 2. Load CIFAR-10

We first load the raw CIFAR-10 data. Each image has shape `(32, 32, 3)`.

[2]:
```
import os
import urllib.request
import tarfile

# Update this path if your local data directory is different.
cifar10_dir = repo_root / "data" / "cifar-10-batches-py"

# Download dataset if it does not exist
if not cifar10_dir.exists():
    print("CIFAR-10 dataset not found. Downloading...")

    data_dir = repo_root / "data"
    data_dir.mkdir(parents=True, exist_ok=True)

    url = "https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz"
    archive_path = data_dir / "cifar-10-python.tar.gz"

    urllib.request.urlretrieve(url, archive_path)

    print("Extracting dataset...")
    with tarfile.open(archive_path, "r:gz") as tar:
        tar.extractall(path=data_dir)

    print("Download complete.")

# Clean up old variables if the notebook has been run before.
for name in ["X_train", "y_train", "X_test", "y_test"]:
    if name in globals():
        del globals()[name]
```

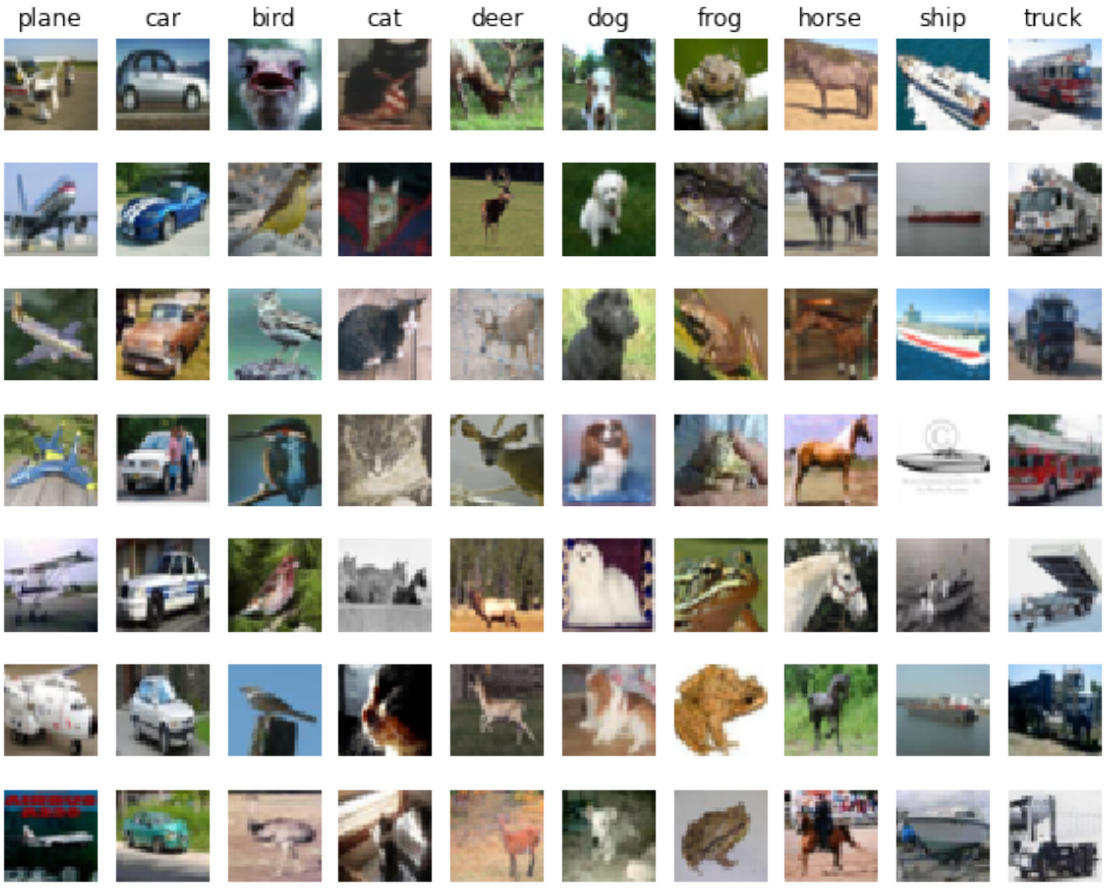
```
# Load CIFAR-10
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

print("Training data shape: ", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Test data shape:     ", X_test.shape)
print("Test labels shape:   ", y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape:      (10000, 32, 32, 3)
Test labels shape:    (10000,)
```

1.6 3. Visualize a few examples

A quick visualization helps confirm both the data layout and the semantic variety across classes.

[3]:
```
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



1.7 4. Subsample and reshape the data

For this demo, we use a smaller subset to keep computation manageable. After subsampling, we flatten each image into a row vector so that k-NN can treat the data as points in Euclidean space.

[4]:
```
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))

print("Flattened training data shape:", X_train.shape)
```

```
print("Flattened test data shape:    ", X_test.shape)
```

```
Flattened training data shape: (5000, 3072)
Flattened test data shape:     (500, 3072)
```

1.8 5. Train the classifier

For k-NN, training is just memorizing the training set.

[5]:
```
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

1.9 6. Compute the distance matrix with two loops

We begin with the most direct implementation. If there are `num_test` test examples and `num_training` training examples, then the distance matrix has shape `(num_test, num_training)`.

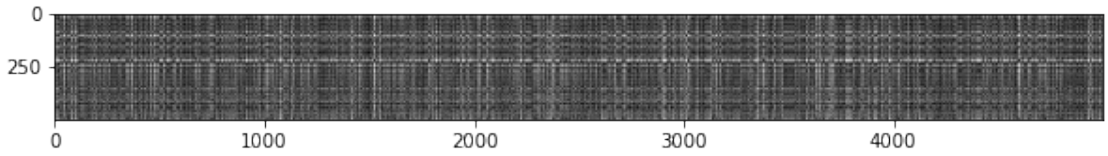
[6]:
```
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

```
(500, 5000)
```

1.10 7. Visualize the distance matrix

Each row corresponds to one test image, and each column corresponds to one training image. Brighter entries indicate larger distances.

[7]:
```
plt.imshow(dists, interpolation='none')
plt.show()
```



1.10.1 Discussion: structured brightness patterns

Two visual patterns often stand out:

- **Bright rows** correspond to test images that are far from many training images. These are often outliers, unusually colored images, or images with a composition not well represented in the training subset.
- **Bright columns** correspond to training images that are far from many test images. These are training-set outliers or unusual examples.

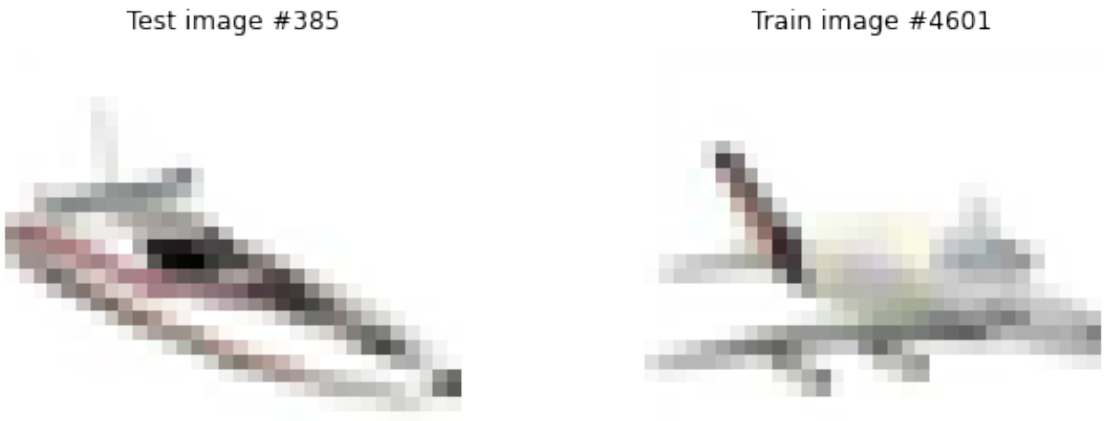
This is a nice reminder that even before classification, the geometry of the data already reveals structure.

1.11 8. Inspect unusually distant examples

The extra inspection below keeps one of the most useful exploratory ideas from the original working notebook: identifying test and training images whose average distances are especially large.

[8]:
```
row_mean = np.mean(dists, axis=1)   # average distance for each test image
col_mean = np.mean(dists, axis=0)   # average distance for each training image

outlier_test_idx = np.argmax(row_mean)
outlier_train_idx = np.argmax(col_mean)

print("Most distant test image index:   ", outlier_test_idx)
print("Most distant training image index:", outlier_train_idx)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(X_test[outlier_test_idx].reshape(32, 32, 3).astype("uint8"))
plt.title(f"Test image #{outlier_test_idx}")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(X_train[outlier_train_idx].reshape(32, 32, 3).astype("uint8"))
plt.title(f"Train image #{outlier_train_idx}")
plt.axis("off")

plt.show()
```

```
Most distant test image index:    385
Most distant training image index: 4601
```



1.12 9. Predict labels from the distance matrix

Once distances are known, prediction is a voting procedure among the nearest neighbors. We start with the simplest choice, `k = 1`.

[9]:
```
y_test_pred = classifier.predict_labels(dists, k=1)

num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print("Got %d / %d correct => accuracy: %f" % (num_correct, num_test, accuracy))
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

For raw-pixel k-NN on CIFAR-10, an accuracy around **27%** is typical here.

[10]:
```
y_test_pred = classifier.predict_labels(dists, k=5)

num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print("Got %d / %d correct => accuracy: %f" % (num_correct, num_test, accuracy))
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

A slightly larger `k` often performs a bit better because it reduces sensitivity to individual noisy neighbors.

1.13 10. Note on preprocessing and distance choice

This demo uses the standard raw-pixel representation together with Euclidean distance.

A useful conceptual question is whether preprocessing always helps k-NN. For example, if one switches to L1 distance, then different normalization choices may interact with the geometry in different ways. There is no universal rule that every centering or scaling scheme improves nearest-neighbor performance; the answer depends on the distance metric and on which variations in the data are actually informative.

1.14 11. Speeding up distance computation

The two-loop implementation is the clearest, but it is slow. We now compare it with partially vectorized and fully vectorized implementations.

[11]:
```
dists_one = classifier.compute_distances_one_loop(X_test)

difference = np.linalg.norm(dists - dists_one, ord='fro')
print("One loop difference was: %f" % difference)
if difference < 0.001:
    print("Good! The distance matrices are the same")
else:
    print("Uh-oh! The distance matrices are different")
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

[12]:
```
dists_two = classifier.compute_distances_no_loops(X_test)

difference = np.linalg.norm(dists - dists_two, ord='fro')
print("No loop difference was: %f" % difference)
if difference < 0.001:
    print("Good! The distance matrices are the same")
else:
    print("Uh-oh! The distance matrices are different")
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

[13]:
```
def time_function(f, *args):
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print("Two loop version took %f seconds" % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print("One loop version took %f seconds" % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print("No loop version took %f seconds" % no_loop_time)
```

```
Two loop version took 22.393625 seconds
One loop version took 10.975750 seconds
No loop version took 0.230714 seconds
```

The fully vectorized version should be much faster. This is one of the central implementation lessons of the notebook: in numerical Python, vectorization matters.

1.15 12. Cross-validation

So far, the choice of `k` has been ad hoc. We now use 5-fold cross-validation on the training set to choose `k` more systematically.

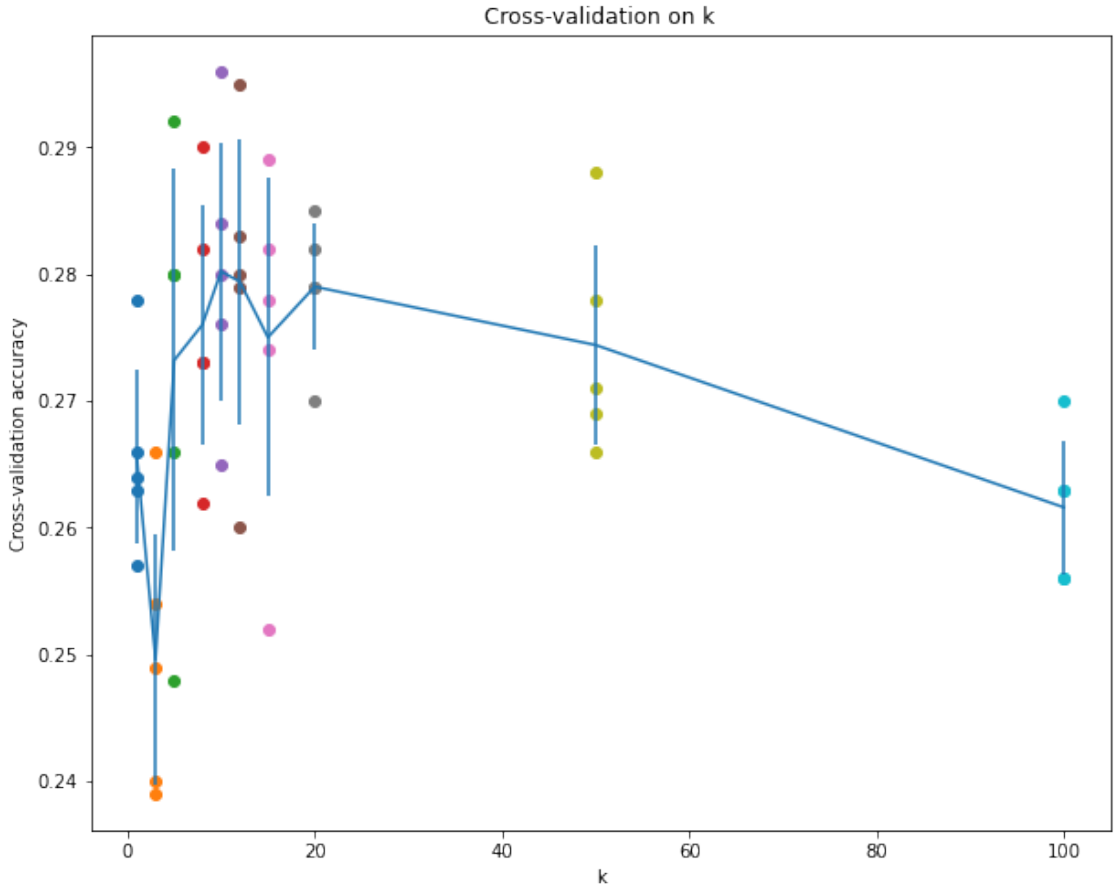
[14]:
```
num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)

k_to_accuracies = {}

for k in k_choices:
```

```
    k_to_accuracies[k] = []
    for i in range(num_folds):
        X_val_fold = X_train_folds[i]
        y_val_fold = y_train_folds[i]

        X_train_cv = np.concatenate([X_train_folds[j] for j in range(num_folds)␣
↪if j != i], axis=0)
        y_train_cv = np.concatenate([y_train_folds[j] for j in range(num_folds)␣
↪if j != i], axis=0)

        classifier = KNearestNeighbor()
        classifier.train(X_train_cv, y_train_cv)

        y_val_pred = classifier.predict(X_val_fold, k=k)
        num_correct = np.sum(y_val_pred == y_val_fold)
        accuracy = float(num_correct) / len(y_val_fold)
        k_to_accuracies[k].append(accuracy)

for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print("k = %d, accuracy = %f" % (k, accuracy))
```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
```

```
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

[15]:
```
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

accuracies_mean = np.array([np.mean(v) for k, v in sorted(k_to_accuracies.
 ↪items())])
accuracies_std = np.array([np.std(v) for k, v in sorted(k_to_accuracies.
 ↪items())])

plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title("Cross-validation on k")
plt.xlabel("k")
plt.ylabel("Cross-validation accuracy")
plt.show()
```



1.16 13. Final evaluation on the test set

We choose the best `k` from cross-validation, retrain on the full training subset, and evaluate once on the held-out test set.

[16]:
```
best_k = 10

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print("Got %d / %d correct => accuracy: %f" % (num_correct, num_test, accuracy))
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

1.17 14. Takeaways

- k-NN is conceptually simple: it stores the training set and predicts by local voting.

- Raw-pixel k-NN is only a modest CIFAR-10 baseline, but it is still valuable pedagogically.
- The notebook illustrates the full supervised learning workflow: data representation, model fitting, validation, testing, and computational tradeoffs.
- The difference between two-loop and vectorized distance computation is a concrete example of why efficient numerical programming matters.

1.18 15. Conceptual checklist

In the original assignment, one of the conceptual checks asked which statements about k-NN are always true.

The correct takeaways are:

1. The decision boundary of k-NN is **not** generally linear.
2. The training error of 1-NN is always less than or equal to that of 5-NN, under the standard convention used here that each training point is its own nearest neighbor.
3. The test error of 1-NN is **not** always lower than that of 5-NN.
4. Prediction time does grow with the size of the training set.

These points are worth keeping because they summarize the main statistical and computational tradeoffs of the method.

02_softmax_repo

April 21, 2026

1 Softmax Classifier on CIFAR-10

This notebook demonstrates a from-scratch implementation of a **multiclass linear classifier trained with the softmax loss** on CIFAR-10.

1.1 Goals

- load and inspect CIFAR-10
- preprocess images into row vectors with a bias term
- implement the softmax loss in both naive and vectorized forms
- verify gradients numerically
- train the classifier with SGD
- tune hyperparameters on a validation set
- visualize the learned class templates

1.2 Related source files

- `src/models/softmax.py`
- `src/models/linear_classifier.py`
- `src/utils/data.py`
- `src/utils/gradient_check.py`

This notebook is written in the same repo-friendly style as the adapted k-NN demo: it is meant to live in `demos/`, while reusable code lives in `src/`.

1.3 0. What is softmax classification?

A linear softmax classifier computes class scores

$$s = xW,$$

where $x \in \mathbf{R}^D$ is one input row vector and $W \in \mathbf{R}^{D \times C}$ is the weight matrix.

These scores are turned into probabilities by

$$p_j = \frac{e^{s_j}}{\sum_k e^{s_k}},$$

and the loss for the correct class y is

$$L = -\log p_y.$$

So the model is still linear in the input, but the softmax loss encourages the correct class score to be large relative to the others.

1.4 1. Setup

The path adjustment below lets the notebook import from the repository root when it is run locally.

The import block first tries a `src/...` layout and then falls back to the original `cs231n/...` layout, so it is convenient while you are still transitioning code.

[1]:
```
import sys
from pathlib import Path

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

import random
import time
import tarfile
import urllib.request

import numpy as np
import matplotlib.pyplot as plt

from src.utils.data import load_CIFAR10
from src.models.softmax import softmax_loss_naive, softmax_loss_vectorized
from src.models.linear_classifier import Softmax
from src.utils.gradient_check import grad_check_sparse
# try:
#     from src.utils.data import load_CIFAR10
#     from src.models.softmax import softmax_loss_naive, softmax_loss_vectorized
#     from src.models.linear_classifier import Softmax
#     from src.utils.gradient_check import grad_check_sparse
#     SOURCE_LAYOUT = "src"
# except ImportError:
#     from cs231n.data_utils import load_CIFAR10
#     from cs231n.classifiers.softmax import softmax_loss_naive,␣
 ↪softmax_loss_vectorized
#     from cs231n.classifiers import Softmax
#     from cs231n.gradient_check import grad_check_sparse
#     SOURCE_LAYOUT = "cs231n"

# print(f"Using source layout: {SOURCE_LAYOUT}")

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

%load_ext autoreload
```

```
%autoreload 2
```

1.5 2. Load CIFAR-10

We first load the raw CIFAR-10 data. Each image has shape `(32, 32, 3)`. If the dataset is missing, the notebook downloads it into `data/`.

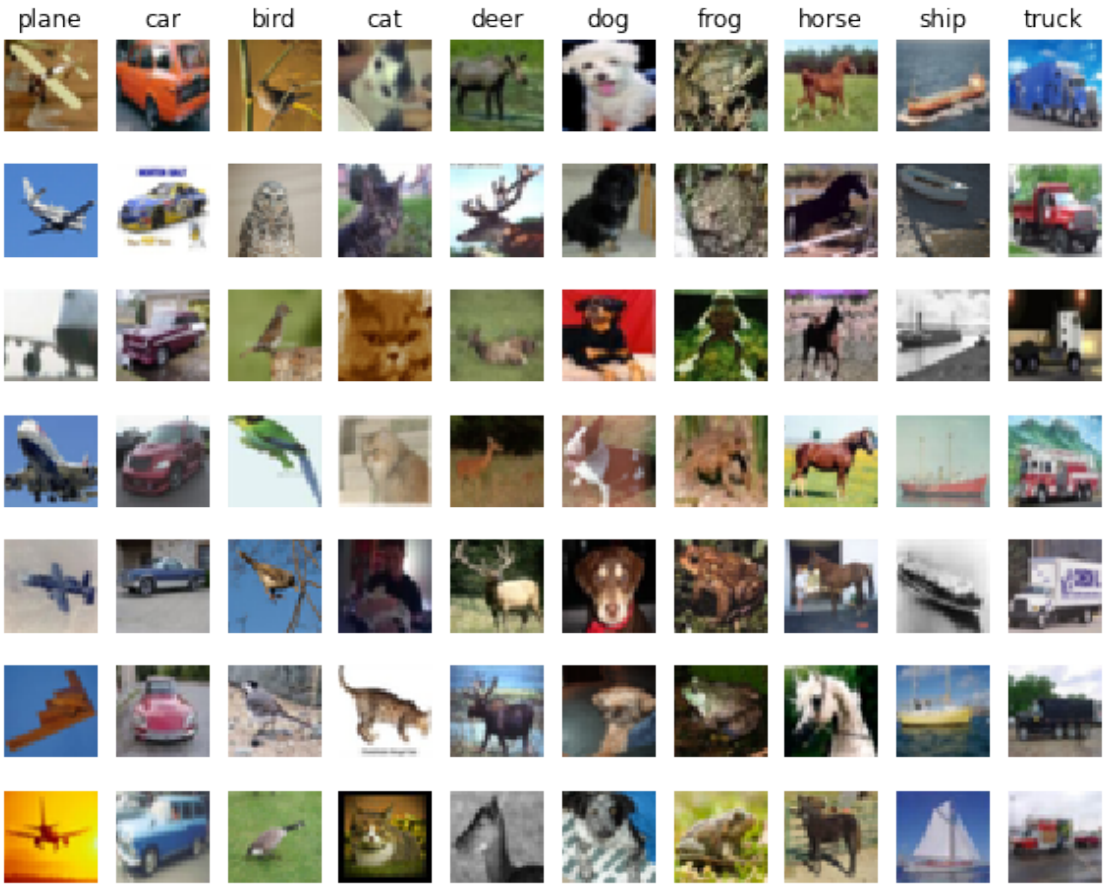
[2]:
```
# Update this path if your local data directory is different.
cifar10_dir = repo_root / "data" / "cifar-10-batches-py"

if not cifar10_dir.exists():
    print("CIFAR-10 dataset not found. Downloading...")

    data_dir = repo_root / "data"
    data_dir.mkdir(parents=True, exist_ok=True)

    url = "https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz"
    archive_path = data_dir / "cifar-10-python.tar.gz"

    urllib.request.urlretrieve(url, archive_path)

    print("Extracting dataset...")
    with tarfile.open(archive_path, "r:gz") as tar:
        tar.extractall(path=data_dir)

    print("Download complete.")

for name in ["X_train", "y_train", "X_test", "y_test"]:
    if name in globals():
        del globals()[name]

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

print("Training data shape: ", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Test data shape:     ", X_test.shape)
print("Test labels shape:   ", y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape: (50000,)
Test data shape:      (10000, 32, 32, 3)
Test labels shape:    (10000,)
```

1.6 3. Visualize a few examples

A quick visualization helps confirm both the data format and the variety across classes.

[3]:
```
classes = ["plane", "car", "bird", "cat", "deer", "dog", "frog", "horse",␣
 ↪"ship", "truck"]
num_classes = len(classes)
samples_per_class = 7

for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype("uint8"))
        plt.axis("off")
        if i == 0:
            plt.title(cls)

plt.show()
```



1.7 4. Split and preprocess the data

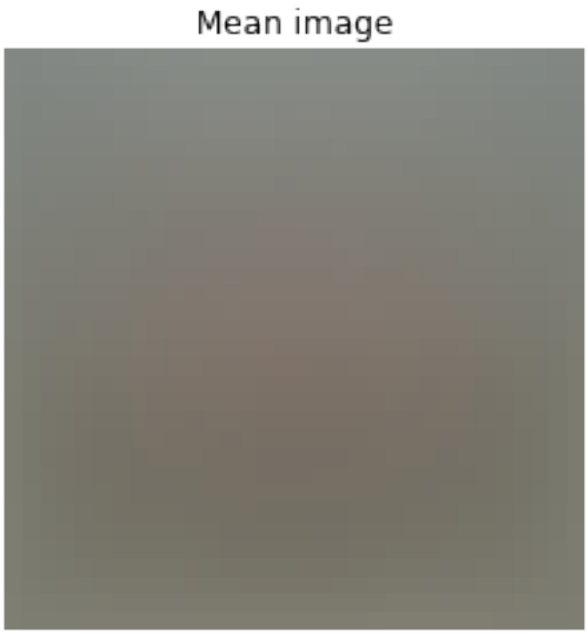
We create: - a training set, - a validation set, - a small development set for fast gradient checks, - and a test set.

Then we: 1. flatten each image into a row vector, 2. subtract the mean image, 3. append a column of ones for the bias trick.

[4]:
```
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

mask = list(range(num_training, num_training + num_validation))
X_val = X_train[mask]
y_val = y_train[mask]

mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

print("Train data shape:      ", X_train.shape)
print("Train labels shape:    ", y_train.shape)
print("Validation data shape: ", X_val.shape)
print("Validation labels:     ", y_val.shape)
print("Test data shape:       ", X_test.shape)
print("Test labels shape:     ", y_test.shape)
print("Dev data shape:        ", X_dev.shape)

X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

print("\nAfter flattening:")
print("X_train:", X_train.shape)
print("X_val:  ", X_val.shape)
print("X_test: ", X_test.shape)
print("X_dev:  ", X_dev.shape)
```

```
mean_image = np.mean(X_train, axis=0)
print("\nFirst 10 entries of mean image:", mean_image[:10])

plt.figure(figsize=(4, 4))
plt.imshow(mean_image.reshape(32, 32, 3).astype("uint8"))
plt.title("Mean image")
plt.axis("off")
plt.show()

X_train = X_train - mean_image
X_val = X_val - mean_image
X_test = X_test - mean_image
X_dev = X_dev - mean_image

X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print("\nAfter mean subtraction and bias trick:")
print("X_train:", X_train.shape)
print("X_val:  ", X_val.shape)
print("X_test: ", X_test.shape)
print("X_dev:  ", X_dev.shape)
```

```
Train data shape:       (49000, 32, 32, 3)
Train labels shape:     (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels:      (1000,)
Test data shape:        (1000, 32, 32, 3)
Test labels shape:      (1000,)
Dev data shape:         (500, 32, 32, 3)

After flattening:
X_train: (49000, 3072)
X_val:   (1000, 3072)
X_test:  (1000, 3072)
X_dev:   (500, 3072)

First 10 entries of mean image: [130.64189796 135.98173469 132.47391837
130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
After mean subtraction and bias trick:
X_train: (49000, 3073)
X_val:   (1000, 3073)
X_test:  (1000, 3073)
X_dev:   (500, 3073)
```

1.8 5. Evaluate the naive softmax loss

We start with a random weight matrix of very small values and evaluate the naive implementation.

[5]:
```
W = np.random.randn(X_train.shape[1], 10) * 1e-4

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e-6)
print("loss: %f" % loss)
print("sanity check: %f" % (-np.log(0.1)))
```

```
loss: 2.379922
sanity check: 2.302585
```

1.8.1 Why should the initial loss be close to $-\log(0.1)$?

With very small random weights, the class scores are all close to each other, so after softmax each class gets probability close to 0.1.

Therefore the correct-class loss for one example is approximately

$$-\log(0.1).$$

This is the multiclass analogue of saying that an untrained classifier is close to making uniform predictions.

1.9 6. Gradient check for the naive implementation

After you implement the gradient in `softmax_loss_naive`, the analytic gradient should match the numerical gradient very closely.

[6]:
```
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: 1.090679 analytic: 1.090679, relative error: 6.615636e-09
numerical: 0.471471 analytic: 0.471471, relative error: 4.345164e-08
numerical: -1.725459 analytic: -1.725459, relative error: 8.333790e-09
numerical: -0.239554 analytic: -0.239554, relative error: 2.012590e-07
numerical: -3.747204 analytic: -3.747204, relative error: 9.003016e-09
numerical: 1.002975 analytic: 1.002975, relative error: 5.699943e-08
numerical: -1.123421 analytic: -1.123421, relative error: 1.868454e-08
numerical: 0.852534 analytic: 0.852534, relative error: 5.387949e-09
numerical: 3.151822 analytic: 3.151822, relative error: 1.950771e-08
numerical: -0.296274 analytic: -0.296274, relative error: 8.021889e-08
numerical: 2.905597 analytic: 2.905596, relative error: 2.933340e-08
numerical: -0.196570 analytic: -0.196570, relative error: 3.793978e-08
numerical: -2.528727 analytic: -2.528727, relative error: 8.787992e-09
numerical: -1.155045 analytic: -1.155045, relative error: 1.369774e-09
numerical: 0.440593 analytic: 0.440593, relative error: 6.702341e-08
numerical: -0.882949 analytic: -0.882949, relative error: 2.837815e-08
numerical: -0.518315 analytic: -0.518315, relative error: 1.243268e-08
numerical: -4.074694 analytic: -4.074694, relative error: 1.043377e-08
numerical: -7.020886 analytic: -7.020886, relative error: 5.070368e-09
numerical: -0.083395 analytic: -0.083395, relative error: 3.408170e-07
```

1.9.1 Note on gradient checking: softmax versus SVM

A useful conceptual distinction is this:

- The **softmax loss** is smooth, so numerical gradient checking is usually extremely stable.
- The **multiclass SVM hinge loss** is not differentiable at margin boundaries, because of the $\max(0, \cdot)$ term.

So if an SVM gradient check occasionally shows a discrepancy in one coordinate, that can happen simply because the finite-difference probe crossed a kink. This is usually **not** a bug by itself.

A one-dimensional toy example is

$$L(w) = \max(0, w).$$

At $w = 0$, the derivative is not uniquely defined, so a symmetric finite-difference estimate can behave awkwardly.

In the multiclass SVM setting, the same issue occurs when some margin

$$s_j - s_y + \Delta$$

is very close to 0.

If the margin parameter Δ is changed, then the locations of those kink boundaries shift. In practice, a larger margin can make it more likely that some sample lies near a hinge boundary, so such rare gradient-check mismatches may appear a bit more often.

1.10 7. Compare naive and vectorized loss

The vectorized implementation should produce the same loss, but much faster.

[7]:
```
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 5e-6)
toc = time.time()
print("Naive loss:      %e computed in %fs" % (loss_naive, toc - tic))

tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 5e-6)
toc = time.time()
print("Vectorized loss: %e computed in %fs" % (loss_vectorized, toc - tic))

print("difference: %f" % (loss_naive - loss_vectorized))
```

```
Naive loss:      2.379922e+00 computed in 0.091852s
Vectorized loss: 2.379922e+00 computed in 0.008136s
difference: -0.000000
```

1.11 8. Compare naive and vectorized gradients

Now we compare the gradients using the Frobenius norm.

[8]:
```
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 5e-6)
toc = time.time()
print("Naive gradient computed in %fs" % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 5e-6)
toc = time.time()
print("Vectorized gradient computed in %fs" % (toc - tic))

difference = np.linalg.norm(grad_naive - grad_vectorized, ord="fro")
```

```
print("gradient difference: %f" % difference)
```

```
Naive gradient computed in 0.089049s
Vectorized gradient computed in 0.003759s
gradient difference: 0.000000
```

1.12 9. Train the classifier with SGD

Once the loss and gradient are correct, we can train the linear classifier by stochastic gradient descent.

The implementation for this part belongs in `src/models/linear_classifier.py` (or the original `cs231n/classifiers/linear_classifier.py` if you are still using the assignment layout).

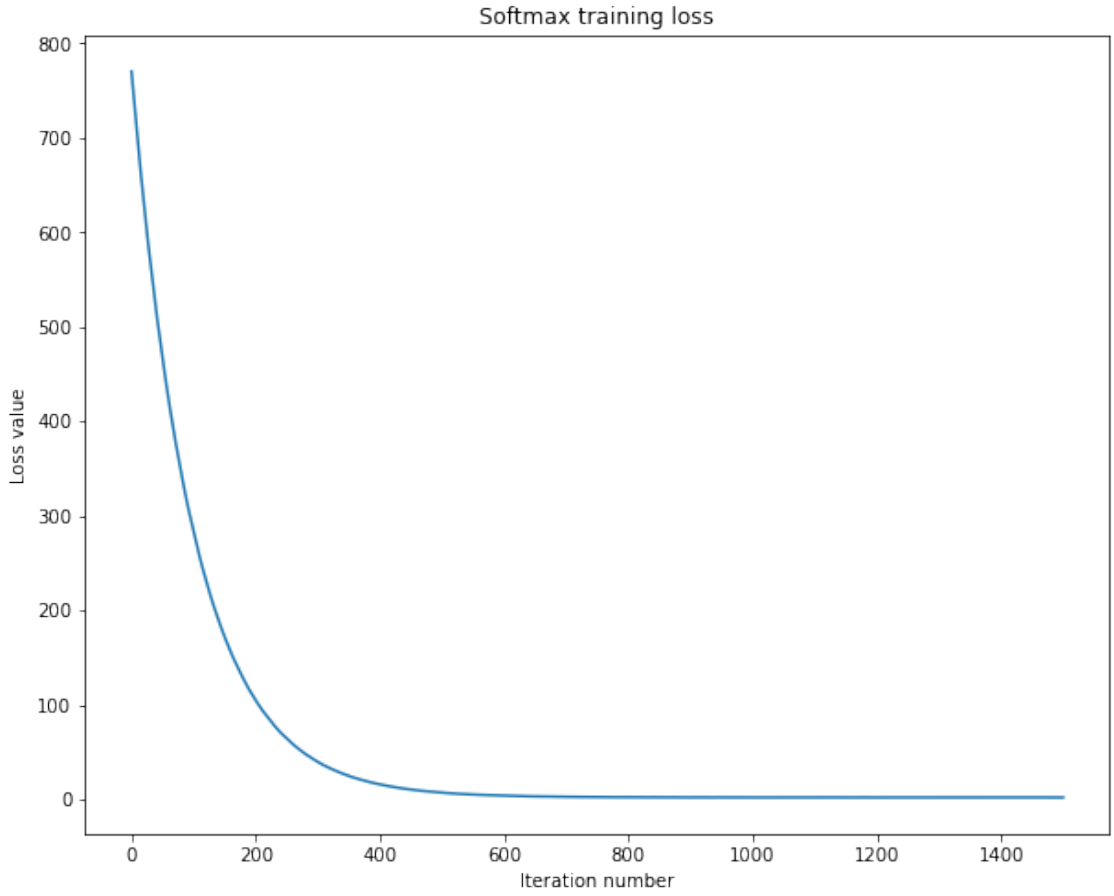
[9]:
```
softmax = Softmax()

tic = time.time()
loss_hist = softmax.train(
    X_train,
    y_train,
    learning_rate=1e-7,
    reg=2.5e4,
    num_iters=1500,
    verbose=True,
)
toc = time.time()

print("Training took %fs" % (toc - tic))
```

```
iteration 0 / 1500: loss 770.262049
iteration 100 / 1500: loss 282.644799
iteration 200 / 1500: loss 104.718728
iteration 300 / 1500: loss 39.681320
iteration 400 / 1500: loss 15.867992
iteration 500 / 1500: loss 7.089083
iteration 600 / 1500: loss 3.920166
iteration 700 / 1500: loss 2.720311
iteration 800 / 1500: loss 2.387500
iteration 900 / 1500: loss 2.118520
iteration 1000 / 1500: loss 2.097127
iteration 1100 / 1500: loss 2.055159
iteration 1200 / 1500: loss 2.079096
iteration 1300 / 1500: loss 2.115100
iteration 1400 / 1500: loss 2.122922
Training took 2.974136s
```

1.13 10. Plot the loss history

A steadily decreasing loss curve is a basic sanity check for SGD.

[10]:
```
plt.plot(loss_hist)
plt.xlabel("Iteration number")
plt.ylabel("Loss value")
plt.title("Softmax training loss")
plt.show()
```



1.14 11. Evaluate on the training and validation sets

A validation accuracy around the low-to-mid 30% range is a reasonable first target here.

[11]:
```
y_train_pred = softmax.predict(X_train)
train_accuracy = np.mean(y_train == y_train_pred)
print("training accuracy:   %f" % train_accuracy)

y_val_pred = softmax.predict(X_val)
val_accuracy = np.mean(y_val == y_val_pred)
print("validation accuracy: %f" % val_accuracy)

artifacts_dir = repo_root / "artifacts"
```

```
artifacts_dir.mkdir(parents=True, exist_ok=True)

softmax.save(str(artifacts_dir / "softmax.npy"))
print(f"Saved model to {artifacts_dir / 'softmax.npy'}")
```

```
training accuracy:   0.329939
validation accuracy: 0.337000
/Users/cao20/cao20Lab/reborn/ml-from-scratch/artifacts/softmax.npy saved.
Saved model to /Users/cao20/cao20Lab/reborn/ml-from-
scratch/artifacts/softmax.npy
```

1.15 12. Hyperparameter tuning

We now search over learning rates and regularization strengths using the validation set.

The grid below is intentionally modest so the notebook is still comfortable to run locally. You can widen it later.

[12]:
```
learning_rates = [5e-8, 1e-7, 2e-7, 5e-7, 1e-6]
regularization_strengths = [1e4, 2.5e4, 5e4, 1e5]

results = {}
best_val = -1.0
best_softmax = None

for lr in learning_rates:
    for reg in regularization_strengths:
        model = Softmax()
        model.train(
            X_train,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=2000,
            verbose=False,
        )

        train_pred = model.predict(X_train)
        val_pred = model.predict(X_val)

        train_acc = np.mean(y_train == train_pred)
        val_acc = np.mean(y_val == val_pred)

        results[(lr, reg)] = (train_acc, val_acc)

        if val_acc > best_val:
            best_val = val_acc
            best_softmax = model
```

```
for (lr, reg), (train_acc, val_acc) in sorted(results.items()):
    print("lr %e reg %e train accuracy: %f val accuracy: %f" % (lr, reg,␣
 ↪train_acc, val_acc))

print("\nbest validation accuracy: %f" % best_val)
```

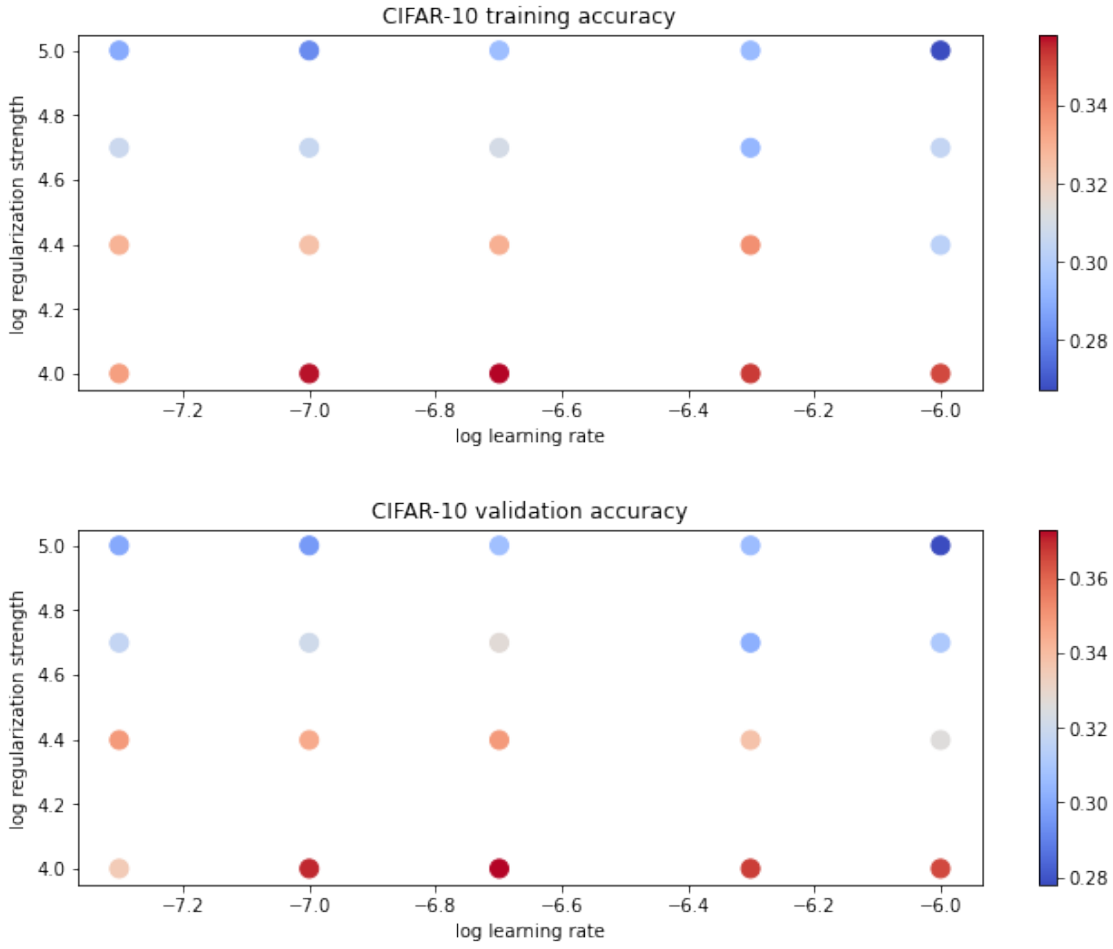
```
lr 5.000000e-08 reg 1.000000e+04 train accuracy: 0.334122 val accuracy: 0.335000
lr 5.000000e-08 reg 2.500000e+04 train accuracy: 0.329327 val accuracy: 0.349000
lr 5.000000e-08 reg 5.000000e+04 train accuracy: 0.306776 val accuracy: 0.317000
lr 5.000000e-08 reg 1.000000e+05 train accuracy: 0.288776 val accuracy: 0.300000
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.356837 val accuracy: 0.370000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.325122 val accuracy: 0.345000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.306000 val accuracy: 0.321000
lr 1.000000e-07 reg 1.000000e+05 train accuracy: 0.280878 val accuracy: 0.296000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.358224 val accuracy: 0.373000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.330061 val accuracy: 0.349000
lr 2.000000e-07 reg 5.000000e+04 train accuracy: 0.309878 val accuracy: 0.327000
lr 2.000000e-07 reg 1.000000e+05 train accuracy: 0.294347 val accuracy: 0.307000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.352673 val accuracy: 0.367000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.337408 val accuracy: 0.338000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.292204 val accuracy: 0.302000
lr 5.000000e-07 reg 1.000000e+05 train accuracy: 0.293286 val accuracy: 0.306000
lr 1.000000e-06 reg 1.000000e+04 train accuracy: 0.350592 val accuracy: 0.365000
lr 1.000000e-06 reg 2.500000e+04 train accuracy: 0.302367 val accuracy: 0.326000
lr 1.000000e-06 reg 5.000000e+04 train accuracy: 0.304612 val accuracy: 0.311000
lr 1.000000e-06 reg 1.000000e+05 train accuracy: 0.267327 val accuracy: 0.278000

best validation accuracy: 0.373000
```

1.16 13. Visualize the hyperparameter search

This reproduces the familiar assignment-style view, but keeps the notebook organized as a repo demo.

[13]:
```
import math

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

marker_size = 100

plt.figure(figsize=(10, 8))

plt.subplot(2, 1, 1)
colors = [results[x][0] for x in results]
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
```

```
plt.xlabel("log learning rate")
plt.ylabel("log regularization strength")
plt.title("CIFAR-10 training accuracy")

plt.subplot(2, 1, 2)
colors = [results[x][1] for x in results]
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel("log learning rate")
plt.ylabel("log regularization strength")
plt.title("CIFAR-10 validation accuracy")

plt.tight_layout(pad=3)
plt.show()
```



1.17 14. Final evaluation on the test set

We evaluate the best validation model once on the held-out test set.

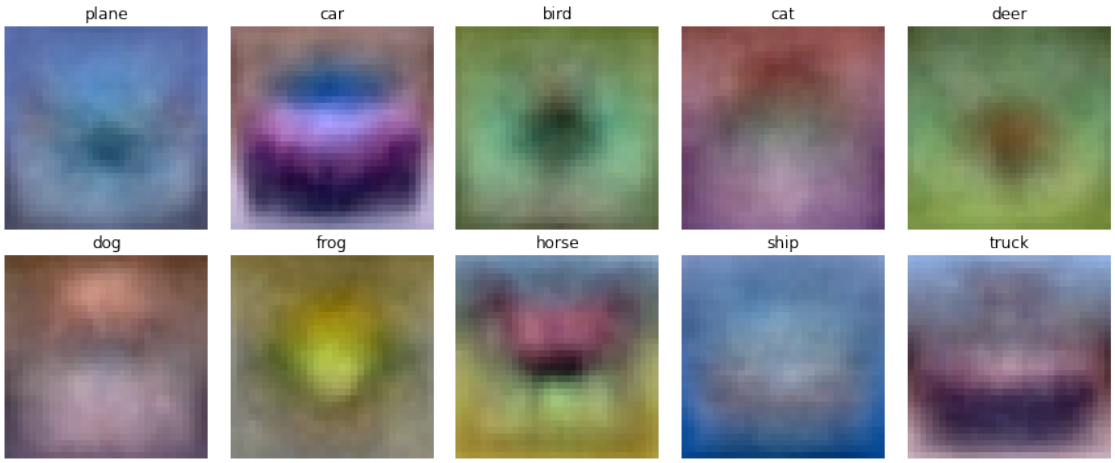
[14]:
```
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print("Softmax classifier on raw pixels final test set accuracy: %f" %␣
 ↪test_accuracy)

best_softmax.save(str(artifacts_dir / "best_softmax.npy"))
print(f"Saved best model to {artifacts_dir / 'best_softmax.npy'}")
```

```
Softmax classifier on raw pixels final test set accuracy: 0.362000
/Users/cao20/cao20Lab/reborn/ml-from-scratch/artifacts/best_softmax.npy saved.
Saved best model to /Users/cao20/cao20Lab/reborn/ml-from-
scratch/artifacts/best_softmax.npy
```

1.18 15. Visualize the learned weights

Each column of the weight matrix can be reshaped back into an image. These visualizations are often interpreted as rough **class templates**.

[15]:
```
w = best_softmax.W[:-1, :]   # remove the bias row
w = w.reshape(32, 32, 3, 10)

w_min, w_max = np.min(w), np.max(w)

plt.figure(figsize=(12, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)

    wimg = 255.0 * (w[:, :, :, i] - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype("uint8"))
    plt.axis("off")
    plt.title(classes[i])

plt.tight_layout()
plt.show()
```



1.18.1 Discussion: what do these weight images mean?

The learned images usually look like blurry, noisy color templates rather than clean objects.

That is expected: - the classifier is **linear**, - it works directly on **raw pixels**, - and each class weight vector is trying to increase the score of one class while decreasing the others.

So each image is not a prototype in the human sense. Instead, it is a map of which pixel patterns push the score up or down for that class. Broad color regions and coarse shape hints often appear, but fine object structure is mostly absent.

1.19 16. Conceptual checklist

Two final takeaways from the original softmax assignment are worth keeping.

1.19.1 Why do the learned weights look blurry?

Because each class weight vector is an average linear template in raw-pixel space. Natural images within one class vary a lot in pose, background, lighting, and exact shape, so the learned template becomes diffuse rather than sharp.

1.19.2 Can a new datapoint change the softmax loss but leave the SVM loss unchanged?

Yes.

If the new datapoint is already classified with a comfortable positive margin, then its multiclass SVM loss is zero, so adding it does not change the total SVM loss.

But the softmax loss is almost never exactly zero, because the softmax still assigns some nonzero probability mass to incorrect classes. So the same datapoint can still contribute a positive amount to the total softmax loss.

This is one of the cleanest conceptual differences between hinge-style losses and probabilistic log-losses.

03_two_layer_net_repo

April 21, 2026

1 Fully-Connected Neural Nets on CIFAR-10

This notebook adapts the original CS231n two-layer network exercise to a **local repo-friendly workflow**.

We keep the assignment's modular, analysis-oriented style: - implement layer-wise forward and backward passes, - verify them with numeric gradient checks, - build a two-layer network, - train it on CIFAR-10 with a solver, - inspect training dynamics and learned first-layer weights, - and discuss common conceptual questions along the way.

1.1 Goals

- work locally from a notebook that lives in `demos/`
- import reusable code from `src/`
- preserve the original debugging and experimentation flow
- keep the assignment checks and training structure intact

1.2 Related source files

- `src/layers.py`
- `src/layer_utils.py`
- `src/models/fc_net.py`
- `src/utils/data.py`
- `src/utils/gradient_check.py`
- `src/utils/solver.py`
- `src/utils/vis_utils.py`

1.3 0. Why a modular fully-connected network?

In this exercise we implement fully-connected networks using a **modular approach**. Each layer has a `forward` function and a `backward` function.

The forward pass receives inputs, weights, and other parameters, computes outputs, and stores a `cache` object containing the intermediate values needed later for backpropagation.

For example, conceptually we want a pattern like

```
def layer_forward(x, w):
    z = ...
    out = ...
```

```
    cache = (x, w, z, out)
    return out, cache
```

and then

```
def layer_backward(dout, cache):
    x, w, z, out = cache
    dx = ...
    dw = ...
    return dx, dw
```

Once several layers are written this way, they can be composed cleanly into larger models with different architectures.

1.4 1. Setup

The notebook is meant to run locally from `demos/`, with reusable implementation code in `src/`.

The setup below: - adds the repository root to `sys.path`, - imports the layer, model, solver, and utility functions from `src/`, - and sets plotting defaults for interactive notebook work.

[10]:
```
from __future__ import print_function

import sys
import time
import tarfile
import urllib.request
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

from src.layers import affine_forward, affine_backward, relu_forward,␣
 ↪relu_backward, softmax_loss
from src.layer_utils import affine_relu_forward, affine_relu_backward
from src.models.fc_net import TwoLayerNet
from src.utils.data import load_CIFAR10
from src.utils.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from src.utils.solver import Solver
from src.utils.vis_utils import visualize_grid

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
```

```
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Return the maximum relative error."""
    return np.max(np.abs(x - y) / np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

1.5 2. Load and preprocess CIFAR-10

The original assignment used a helper that returned a preprocessed CIFAR-10 dictionary. Here we reproduce the same local workflow explicitly:

1. load raw CIFAR-10 from `data/`,
2. download it automatically if it is missing,
3. split into training / validation / test subsets,
4. subtract the mean training image,
5. transpose images to channel-first format `(N, 3, 32, 32)`.

This keeps the later assignment cells unchanged in spirit while making the notebook self-contained for local use.

[11]:
```
# Update this path if your local data directory is different.
cifar10_dir = repo_root / "data" / "cifar-10-batches-py"

if not cifar10_dir.exists():
    print("CIFAR-10 dataset not found. Downloading...")

    data_dir = repo_root / "data"
    data_dir.mkdir(parents=True, exist_ok=True)

    url = "https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz"
    archive_path = data_dir / "cifar-10-python.tar.gz"

    urllib.request.urlretrieve(url, archive_path)

    print("Extracting dataset...")
    with tarfile.open(archive_path, "r:gz") as tar:
        tar.extractall(path=data_dir)

    print("Download complete.")

def get_CIFAR10_data_local(
    num_training=49000,
    num_validation=1000,
    num_test=1000,
```

```
    subtract_mean=True,
):
    """Match the original assignment preprocessing, but using the local repo␣
 ↪layout."""
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]

    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]

    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    if subtract_mean:
        mean_image = np.mean(X_train, axis=0)
        X_train = X_train - mean_image
        X_val = X_val - mean_image
        X_test = X_test - mean_image

    X_train = X_train.transpose(0, 3, 1, 2).copy()
    X_val = X_val.transpose(0, 3, 1, 2).copy()
    X_test = X_test.transpose(0, 3, 1, 2).copy()

    return {
        "X_train": X_train,
        "y_train": y_train,
        "X_val": X_val,
        "y_val": y_val,
        "X_test": X_test,
        "y_test": y_test,
    }

data = get_CIFAR10_data_local()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.6 3. Affine layer: forward

Open `src/layers.py` and implement `affine_forward`.

Once that is done, the following numerical check should confirm that the implementation is behaving correctly.

[12]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),␣
 ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.7698500479884e-10
```

1.7 4. Affine layer: backward

Now implement `affine_backward` and test it using numeric gradient checking.

[13]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,␣
 ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,␣
 ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,␣
 ↪dout)
```

```
_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  1.0908199508708189e-10
dw error:  2.1752635504596857e-10
db error:  7.736978834487815e-12
```

1.8 5. ReLU activation: forward

Implement the forward pass for ReLU in `relu_forward`, then run the test below.

[14]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

1.9 6. ReLU activation: backward

Now implement `relu_backward` and verify it with numeric gradient checking.

[15]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
```

```
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

1.9.1 Discussion: activation functions and gradient flow

We've only implemented ReLU in code here, but it is useful to compare it with other common activation functions from a backpropagation point of view.

Question. Which of the following can suffer from zero or near-zero gradient flow, and for what kinds of inputs?

1. Sigmoid
2. ReLU
3. Leaky ReLU

Answer.

- **Sigmoid:** yes.

  $$\sigma(x) = \frac{1}{1+e^{-x}}, \qquad \sigma'(x) = \sigma(x)(1-\sigma(x)).$$

  When $x \gg 0$, we have $\sigma(x) \approx 1$, and when $x \ll 0$, we have $\sigma(x) \approx 0$.
  In both cases, $\sigma'(x) \approx 0$, so the gradient becomes very small. This is the classic saturation problem.

- **ReLU:** yes.

  $$\mathrm{ReLU}(x) = \max(0, x), \qquad \mathrm{ReLU}'(x) = \begin{cases} 1, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

  So the gradient is exactly zero on the negative half-line. If a neuron stays in that region, it can become a "dead" ReLU.

- **Leaky ReLU:** much less so.

  $$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \leq 0, \end{cases} \qquad \mathrm{LeakyReLU}'(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x \leq 0, \end{cases}$$

  where $\alpha > 0$ is small.
  The gradient on the negative side is small but still nonzero, so gradient flow is usually better preserved.

In short: **sigmoid** can have near-zero gradients for large positive or negative inputs, **ReLU** has exactly zero gradient for negative inputs, and **Leaky ReLU** is designed to reduce that problem.

1.10 7. Convenience "sandwich" layers

Certain layer patterns appear repeatedly in fully-connected networks. A common example is an affine transformation followed immediately by a ReLU.

To make these patterns easy to reuse, we define convenience layers in `src/layer_utils.py`.

For now, inspect `affine_relu_forward` and `affine_relu_backward`, then run the gradient check below.

[16]:
```
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,␣
 ↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,␣
 ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,␣
 ↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  6.395535042049294e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

1.11 8. Loss layer: softmax

Implement the loss and gradient for `softmax_loss` in `src/layers.py`.

This should feel very similar to the softmax work from the earlier classifier notebook; the main difference is that now the loss is treated as one modular layer inside a larger network.

[17]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)
```

```
dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,␣
 ↪verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should␣
 ↪be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.3025458445007376
dx error:  8.234144091578429e-09
```

1.12 9. Two-layer network

Open `src/models/fc_net.py` and complete the implementation of `TwoLayerNet`.

Read through the class carefully so that the API and the parameter shapes are clear, then run the checks below.

[18]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
```

```
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.
 ↪33206765,  16.09215096],
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.
 ↪49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.
 ↪66781506,  16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization …
Testing test-time forward pass …
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.22e-08
W2 relative error: 3.34e-10
b1 relative error: 4.73e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 1.37e-07
b1 relative error: 1.56e-08
b2 relative error: 9.09e-10
```

1.13 10. Train with the solver

Read through `src/utils/solver.py` to familiarize yourself with the training API.

Then use a `Solver` instance to train a `TwoLayerNet` that reaches roughly **36% validation accuracy**.

[19]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None

################################################################################
# TODO: Use a Solver instance to train a TwoLayerNet that achieves about 36% #
# accuracy on the validation set.                                              #
################################################################################
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3,
    },
    lr_decay=0.95,
    num_epochs=10,
    batch_size=200,
    print_every=100,
    verbose=True
)
solver.train()
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
(Iteration 1 / 2450) loss: 2.301725
(Epoch 0 / 10) train acc: 0.187000; val_acc: 0.181000
(Iteration 101 / 2450) loss: 1.840451
(Iteration 201 / 2450) loss: 1.744794
(Epoch 1 / 10) train acc: 0.383000; val_acc: 0.418000
(Iteration 301 / 2450) loss: 1.529679
(Iteration 401 / 2450) loss: 1.543941
(Epoch 2 / 10) train acc: 0.444000; val_acc: 0.439000
(Iteration 501 / 2450) loss: 1.552313
(Iteration 601 / 2450) loss: 1.511572
(Iteration 701 / 2450) loss: 1.462240
(Epoch 3 / 10) train acc: 0.476000; val_acc: 0.444000
(Iteration 801 / 2450) loss: 1.515084
(Iteration 901 / 2450) loss: 1.462425
(Epoch 4 / 10) train acc: 0.495000; val_acc: 0.469000
(Iteration 1001 / 2450) loss: 1.376394
(Iteration 1101 / 2450) loss: 1.612014
```

```
(Iteration 1201 / 2450) loss: 1.612220
(Epoch 5 / 10) train acc: 0.509000; val_acc: 0.477000
(Iteration 1301 / 2450) loss: 1.427094
(Iteration 1401 / 2450) loss: 1.422479
(Epoch 6 / 10) train acc: 0.535000; val_acc: 0.480000
(Iteration 1501 / 2450) loss: 1.395185
(Iteration 1601 / 2450) loss: 1.405752
(Iteration 1701 / 2450) loss: 1.210371
(Epoch 7 / 10) train acc: 0.543000; val_acc: 0.508000
(Iteration 1801 / 2450) loss: 1.204663
(Iteration 1901 / 2450) loss: 1.364992
(Epoch 8 / 10) train acc: 0.524000; val_acc: 0.488000
(Iteration 2001 / 2450) loss: 1.294265
(Iteration 2101 / 2450) loss: 1.311599
(Iteration 2201 / 2450) loss: 1.323174
(Epoch 9 / 10) train acc: 0.575000; val_acc: 0.509000
(Iteration 2301 / 2450) loss: 1.209796
(Iteration 2401 / 2450) loss: 1.320206
(Epoch 10 / 10) train acc: 0.528000; val_acc: 0.504000
```

1.14 11. Debug the training

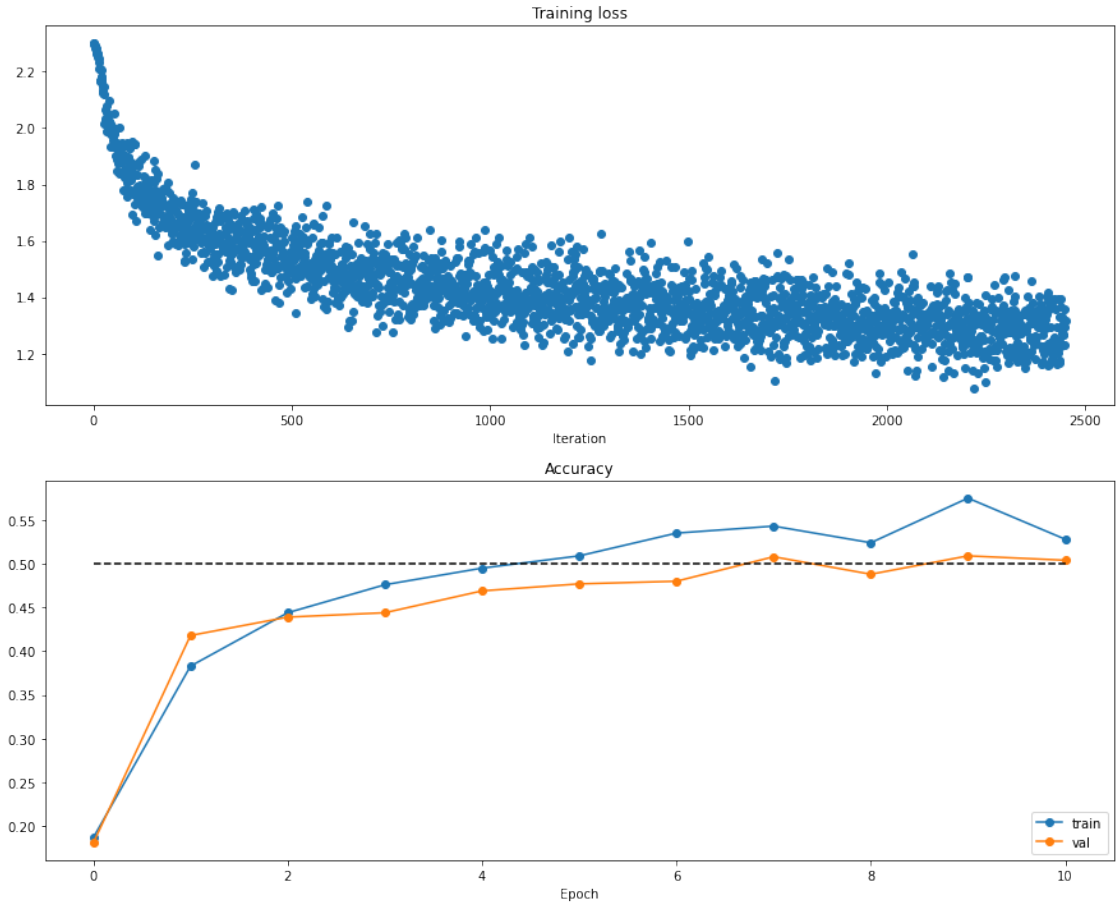
With the default hyperparameters above, the model usually reaches validation accuracy around `0.36`, which is not especially good.

Two standard debugging moves are helpful here:

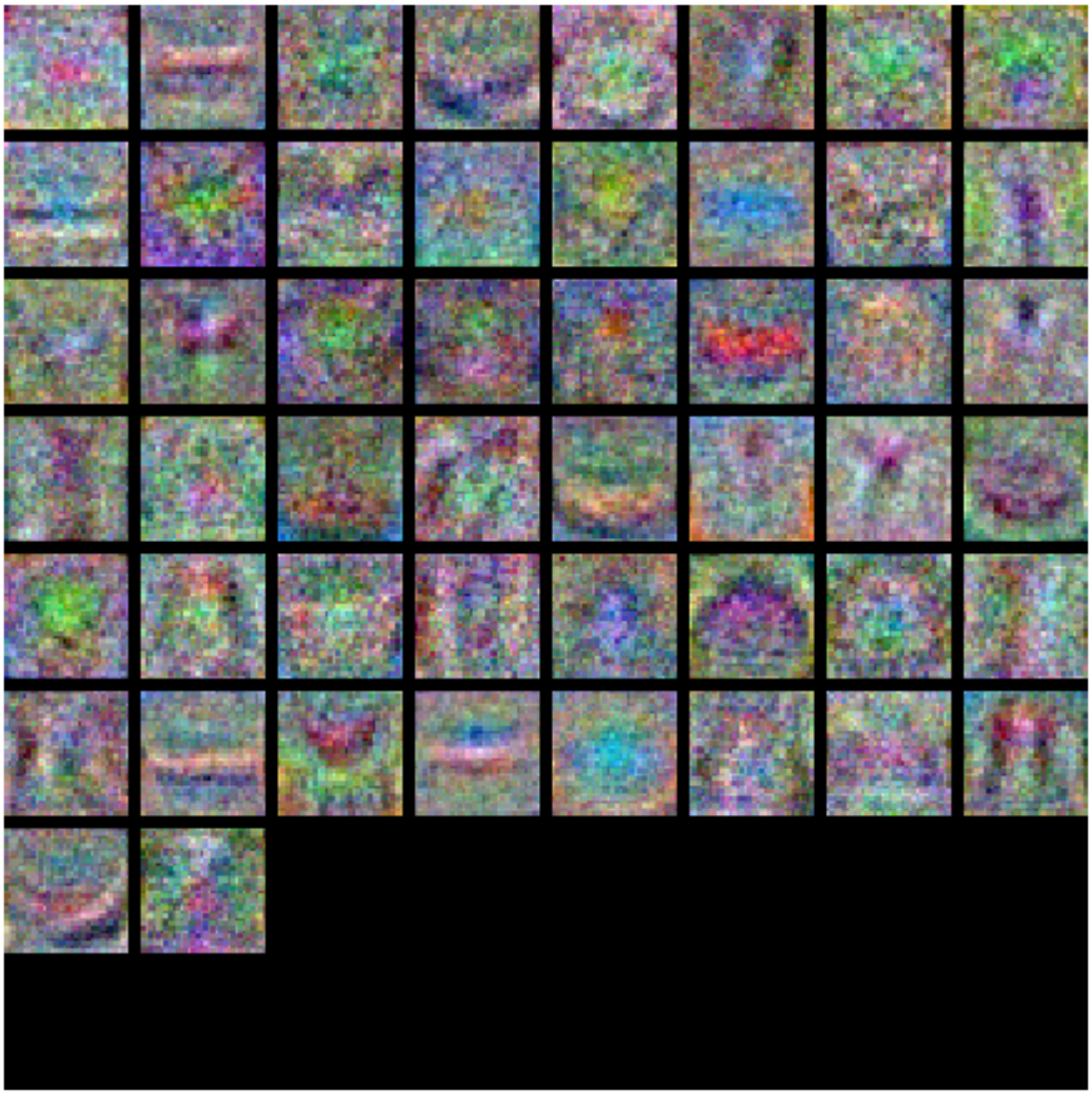
1. plot the loss and train/validation accuracies over time,
2. visualize the first-layer weights.

The first can reveal optimization issues, while the second can show whether the network is learning any meaningful low-level visual structure.

[20]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```



[21]:
```
# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



1.15 12. Tune the hyperparameters

What might be going wrong?
If the loss decreases only slowly and almost linearly, that often suggests the learning rate is too small. If there is little or no gap between training and validation accuracy, that suggests the model may have too little capacity.

On the other hand, if the model becomes too large, then overfitting can become more severe, which would show up as a widening train/validation gap.

Tuning practice.
A large part of neural-network work is developing intuition for hyperparameters. Here you should experiment with: - hidden layer size, - learning rate, - number of epochs, - regularization strength, - and possibly learning-rate decay.

A reasonable target is **above 48% validation accuracy**. Reference implementations can exceed 52%, but the main goal here is to build practical intuition through experimentation.

[22]:
```
best_model = None
best_val_acc = -1
best_stats = None

################################################################################
# TODO: Tune hyperparameters using the validation set. Store your best trained #
# model in best_model.                                                         #
################################################################################

hidden_sizes = [100, 200]
learning_rates = [1e-3, 3e-3]
regularization_strengths = [0.25, 0.5]

for hs in hidden_sizes:
    for lr in learning_rates:
        for reg in regularization_strengths:

            model = TwoLayerNet(input_size, hs, num_classes, reg=reg)

            solver = Solver(
                model,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=10,
                batch_size=200,
                print_every=100,
                verbose=False
            )

            solver.train()

            val_acc = solver.best_val_acc
            train_acc = solver.train_acc_history[-1]

            print(
                f"hs={hs}, lr={lr}, reg={reg}, "
                f"train_acc={train_acc:.4f}, val_acc={val_acc:.4f}"
            )

            if val_acc > best_val_acc:
```

```
                best_val_acc = val_acc
                best_model = model
                best_stats = {
                    'hidden_size': hs,
                    'learning_rate': lr,
                    'reg': reg,
                    'train_acc': train_acc,
                    'val_acc': val_acc,
                }

print("best_val_acc:", best_val_acc)
print("best_stats:", best_stats)

################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
hs=100, lr=0.001, reg=0.25, train_acc=0.5800, val_acc=0.5160
hs=100, lr=0.001, reg=0.5, train_acc=0.5620, val_acc=0.5160
hs=100, lr=0.003, reg=0.25, train_acc=0.5480, val_acc=0.4910
hs=100, lr=0.003, reg=0.5, train_acc=0.4560, val_acc=0.4960
hs=200, lr=0.001, reg=0.25, train_acc=0.6090, val_acc=0.5350
hs=200, lr=0.001, reg=0.5, train_acc=0.5990, val_acc=0.5350
hs=200, lr=0.003, reg=0.25, train_acc=0.5790, val_acc=0.5030
hs=200, lr=0.003, reg=0.5, train_acc=0.5340, val_acc=0.5130
best_val_acc: 0.535
best_stats: {'hidden_size': 200, 'learning_rate': 0.001, 'reg': 0.25,
'train_acc': 0.609, 'val_acc': 0.535}
```

1.16 13. Final evaluation

Run the best model on the validation and test sets. A good two-layer network here should exceed roughly **48% validation accuracy**.

[23]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.535
```

[24]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.535
```

[26]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

1.16.1 Discussion: how can we reduce a train-test gap?

After training a neural network, it is common to see training accuracy noticeably higher than test accuracy. That gap is a sign of **overfitting**.

Question. Which of the following can help reduce that gap?

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Answer: 1 and 3

Explanation.

- **Train on a larger dataset:** yes. More data makes it harder for the model to memorize idiosyncrasies of the training set and usually improves generalization.
- **Add more hidden units:** usually no. This increases model capacity, which often makes overfitting worse rather than better.
- **Increase regularization strength:** yes. Stronger regularization discourages overly large or overly flexible weights and can reduce overfitting.
- **None of the above:** incorrect, since both **1** and **3** address the issue directly.

04_features_repo

April 21, 2026

1 Image Features on CIFAR-10

This notebook adapts the original CS231n image-features exercise to the **local repo-friendly workflow** used by the other notebooks in `demos/`.

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of the work is still done in this notebook, but the code path is adapted for this repo: - work locally from a notebook in `demos/` - reuse implementations from `src/` - keep the original feature-engineering workflow intact - compare a linear model and a shallow neural network on the same feature representation

1.1 Related source files

- `src/features.py`
- `src/models/linear_classifier.py`
- `src/models/fc_net.py`
- `src/utils/data.py`
- `src/utils/solver.py`

1.2 0. Why use hand-crafted image features?

Raw pixels are easy to feed into a classifier, but they are not always the most informative representation. In this notebook we combine two classic feature families:

- **HOG (Histogram of Oriented Gradients)** to emphasize local edge and texture structure,
- **HSV hue histograms** to summarize coarse color information.

The point is not that these features beat modern deep vision models. The point is that they make the representation-design question concrete: **better inputs can make simple models much stronger**.

1.3 1. Setup

The setup below mirrors the other repo notebooks: - add the repository root to `sys.path`, - import local code from `src/`, - create output directories used by the model save helpers, - and configure plotting for notebook work.

[ ]:
```
import sys
from pathlib import Path

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

import random
import tarfile
import urllib.request

import matplotlib.pyplot as plt
import numpy as np

from src.features import color_histogram_hsv, extract_features, hog_feature
from src.models.fc_net import TwoLayerNet
from src.models.linear_classifier import Softmax
from src.utils.data import load_CIFAR10
from src.utils.solver import Solver

random.seed(231)
np.random.seed(231)

(repo_root / 'artifacts').mkdir(parents=True, exist_ok=True)

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

%load_ext autoreload
%autoreload 2
```

1.4 2. Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

The only repo-specific difference is that we load it from the local `data/` directory instead of the original course layout, and keep the images in `(N, 32, 32, 3)` format since the feature extractors operate on standard HWC image arrays.

[ ]:
```
cifar10_dir = repo_root / 'data' / 'cifar-10-batches-py'
```

```
if not cifar10_dir.exists():
    print('CIFAR-10 dataset not found. Downloading...')

    data_dir = repo_root / 'data'
    data_dir.mkdir(parents=True, exist_ok=True)

    url = 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz'
    archive_path = data_dir / 'cifar-10-python.tar.gz'

    urllib.request.urlretrieve(url, archive_path)

    print('Extracting dataset...')
    with tarfile.open(archive_path, 'r:gz') as tar:
        tar.extractall(path=data_dir)

    print('Download complete.')

for name in ['X_train', 'y_train', 'X_test', 'y_test']:
    if name in globals():
        del globals()[name]

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

print('Training data shape: ', X_train.shape)
print('Training labels shape:', y_train.shape)
print('Test data shape:     ', X_test.shape)
print('Test labels shape:   ', y_test.shape)
```

1.5 3. Split the data

We follow the same train / validation / test split used in the original assignment: - 49,000 training images, - 1,000 validation images, - 1,000 test images.

The validation split is what we will use for hyperparameter search.

[ ]:
```
num_training = 49000
num_validation = 1000
num_test = 1000

mask = list(range(num_training, num_training + num_validation))
X_val = X_train[mask]
y_val = y_train[mask]

mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

mask = list(range(num_test))
```

```
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape:      ', X_train.shape)
print('Train labels shape:    ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels:     ', y_val.shape)
print('Test data shape:       ', X_test.shape)
print('Test labels shape:     ', y_test.shape)
```

1.6 4. Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The `extract_features` function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each row is the concatenation of all feature vectors for a single image.

In this repo version those helpers live in `src/features.py`, but the modeling idea is unchanged from the original notebook.

[ ]:
```
num_color_bins = 25
feature_fns = [
    hog_feature,
    lambda img: color_histogram_hsv(img, nbin=num_color_bins),
]

X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

std_feat = np.std(X_train_feats, axis=0, keepdims=True)
std_feat[std_feat < 1e-8] = 1.0
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat
```

```
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])

print('Feature matrix shapes:')
print('X_train_feats:', X_train_feats.shape)
print('X_val_feats:  ', X_val_feats.shape)
print('X_test_feats: ', X_test_feats.shape)
```

1.6.1 Why normalize the feature coordinates?

The HOG block and the color-histogram block naturally live on different scales. Without normalization, optimization can be dominated by whichever coordinates happen to have the largest magnitudes. Standardizing the features makes the learning-rate and regularization search much more stable.

1.7 5. Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

The important point here is that the model family is still linear in the feature vector. The expected improvement comes from the representation, not from making the classifier itself more expressive.

[ ]:
```
learning_rates = [1e-7, 5e-7, 1e-6]
regularization_strengths = [5e5, 1e6, 5e6]

results = {}
best_val = -1.0
best_softmax = None
best_softmax_stats = None

for lr in learning_rates:
    for reg in regularization_strengths:
        model = Softmax()
        model.train(
            X_train_feats,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=2000,
            batch_size=200,
            verbose=False,
        )

        train_pred = model.predict(X_train_feats)
```

```
        val_pred = model.predict(X_val_feats)

        train_acc = np.mean(y_train == train_pred)
        val_acc = np.mean(y_val == val_pred)
        results[(lr, reg)] = (train_acc, val_acc)

        if val_acc > best_val:
            best_val = val_acc
            best_softmax = model
            best_softmax_stats = {
                'learning_rate': lr,
                'reg': reg,
                'train_acc': train_acc,
                'val_acc': val_acc,
            }

for (lr, reg), (train_acc, val_acc) in sorted(results.items()):
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (lr, reg,␣
 ↪train_acc, val_acc))

print('\nbest validation accuracy:', best_val)
print('best_softmax_stats:', best_softmax_stats)
```

1.8 6. Final Softmax evaluation

A reasonably tuned model should comfortably outperform the raw-pixel linear baselines and typically land around or above **42% test accuracy**.

[ ]:
```
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax test accuracy:', test_accuracy)
```

[ ]:
```
# Save best softmax model
best_softmax.save('best_softmax_features.npy')
```

1.9 7. Visualize misclassifications

Looking at mistakes is often more informative than looking at a single scalar accuracy. The visualization below groups false positives by predicted class so that we can see which visual confusions recur.

[ ]:
```
examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']

for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    if len(idxs) == 0:
```

```
        continue
    sample_size = min(examples_per_class, len(idxs))
    idxs = np.random.choice(idxs, sample_size, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +␣
↪1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)

plt.show()
```

1.9.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer:

The misclassifications often involve classes with similar global features. For example, trucks and cars are frequently confused because they share similar HOG edge structure and sometimes similar color distributions. Birds may also be misclassified as planes when the background is blue or when the silhouette is elongated in a way that HOG treats similarly. These mistakes make sense because the hand-crafted features focus on texture and coarse color information, but do not capture higher-level semantic parts as effectively as deeper learned representations.

1.9.2 Discussion: what mistakes should we expect?

The most common confusions usually involve classes that overlap in either: - **global shape** as seen by HOG, or - **dominant color distribution** as seen by the hue histogram.

So it is reasonable to see confusions like: - cars vs trucks, - birds vs planes against blue backgrounds, - deer vs horses in natural scenes.

Those errors make sense because these hand-crafted features capture edges, texture, and coarse color, but they do not encode higher-level semantic parts as effectively as deeper learned representations.

1.10 8. Train a two-layer network on the same features

Earlier in the assignment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; strong runs can get close to 60%.

In the repo version we remove the bias column added for the linear classifier and then train a `TwoLayerNet` on the standardized feature vectors themselves.

[ ]:
```
X_train_feats_nn = X_train_feats[:, :-1].copy()
X_val_feats_nn = X_val_feats[:, :-1].copy()
X_test_feats_nn = X_test_feats[:, :-1].copy()

print('Neural-net feature shapes:')
print('X_train_feats_nn:', X_train_feats_nn.shape)
print('X_val_feats_nn:  ', X_val_feats_nn.shape)
print('X_test_feats_nn: ', X_test_feats_nn.shape)
```

[ ]:
```
input_dim = X_train_feats_nn.shape[1]
num_classes = 10

data = {
    'X_train': X_train_feats_nn,
    'y_train': y_train,
    'X_val': X_val_feats_nn,
    'y_val': y_val,
    'X_test': X_test_feats_nn,
    'y_test': y_test,
}

hidden_dims = [500, 1000]
learning_rates = [0.5, 1.0]
regularization_strengths = [1e-4, 1e-3]

best_net = None
best_net_stats = None
best_net_val = -1.0

for hidden_dim in hidden_dims:
    for lr in learning_rates:
        for reg in regularization_strengths:
            net = TwoLayerNet(
                input_dim=input_dim,
                hidden_dim=hidden_dim,
                num_classes=num_classes,
                weight_scale=1e-3,
                reg=reg,
            )
            solver = Solver(
                net,
                data,
                update_rule='sgd',
                optim_config={'learning_rate': lr},
                lr_decay=0.95,
                num_epochs=10,
                batch_size=200,
```

```
                print_every=100,
                verbose=False,
            )
            solver.train()

            train_acc = solver.train_acc_history[-1]
            val_acc = solver.best_val_acc

            print(
                f'hidden_dim={hidden_dim}, lr={lr}, reg={reg}, '
                f'train_acc={train_acc:.4f}, val_acc={val_acc:.4f}'
            )

            if val_acc > best_net_val:
                best_net_val = val_acc
                best_net = net
                best_net_stats = {
                    'hidden_dim': hidden_dim,
                    'learning_rate': lr,
                    'reg': reg,
                    'train_acc': train_acc,
                    'val_acc': val_acc,
                }

print('\nbest two-layer validation accuracy:', best_net_val)
print('best_net_stats:', best_net_stats)
```

1.11 9. Final neural-network evaluation

A tuned two-layer network on top of these features should usually beat the linear feature classifier and often reach **58%+ test accuracy**.

[ ]:
```
y_val_pred = np.argmax(best_net.loss(data['X_val']), axis=1)
y_test_pred_nn = np.argmax(best_net.loss(data['X_test']), axis=1)

print('Two-layer validation accuracy:', np.mean(y_val_pred == data['y_val']))
print('Two-layer test accuracy:      ', np.mean(y_test_pred_nn ==␣
 ↪data['y_test']))
```

[ ]:
```
# Save best two-layer model
best_net.save('best_two_layer_net_features.npy')
```

1.12 10. Takeaways

We have now compared two different model families on the same hand-crafted feature representation.

- A better representation can dramatically improve a simple classifier.

- HOG and hue histograms capture useful low-level structure that raw pixels hide.
- A shallow nonlinear network on top of engineered features can improve performance further.
- Inspecting mistakes is essential: accuracy tells us how much error remains, while qualitative plots help explain why.

This is one of the main lessons of the notebook: even before using deeper models, the choice of representation can change the quality of the classifier substantially.

05_fully_connected_nets_repo

April 21, 2026

1 Multi-Layer Fully Connected Networks on CIFAR-10

This notebook adapts the original CS231n `FullyConnectedNets.ipynb` exercise to the **local repo-friendly workflow** used by the other notebooks in `demos/`.

We keep the assignment's experimentation flow, but make it work directly with this repository: - run locally from `demos/` - import reusable implementations from `src/` - load CIFAR-10 from the repo's `data/` directory - preserve the original gradient checks, optimizer experiments, and model-tuning tasks

1.1 Related source files

- `src/models/fc_net.py`
- `src/layers.py`
- `src/optim.py`
- `src/utils/data.py`
- `src/utils/gradient_check.py`
- `src/utils/solver.py`

1.2 0. Why deeper fully connected networks?

A two-layer network is already expressive enough to learn nonlinear decision boundaries, but deeper fully connected networks let us compose several affine + nonlinearity blocks into a richer function class.

That extra depth introduces two practical challenges that this notebook focuses on: - **implementation discipline**: every layer must expose a clean forward and backward interface; - **optimization stability**: deeper networks are more sensitive to initialization and update rules.

The assignment is structured around those two ideas: first make the modular implementation correct, then make optimization work well enough to train a useful classifier on CIFAR-10.

1.3 1. Setup

The setup below mirrors the earlier repo notebooks: - add the repository root to `sys.path`, - import the local model, solver, and gradient-check helpers from `src/`, - create the `artifacts/` directory for saved models, - and configure plotting defaults for notebook work.

[1]:
```
from __future__ import print_function
```

```
import sys
import tarfile
import urllib.request
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

repo_root = Path().resolve().parent
if str(repo_root) not in sys.path:
    sys.path.append(str(repo_root))

from src.models.fc_net import FullyConnectedNet
from src.utils.data import load_CIFAR10
from src.utils.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from src.utils.solver import Solver

(repo_root / 'artifacts').mkdir(parents=True, exist_ok=True)

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

%load_ext autoreload
%autoreload 2


def rel_error(x, y):
    """Return the maximum relative error."""
    return np.max(np.abs(x - y) / np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

1.4 2. Load and preprocess CIFAR-10

The original notebook used `get_CIFAR10_data()` from the course codebase. Here we reproduce the same preprocessing locally:

1. load raw CIFAR-10 from `data/`,
2. download it automatically if it is missing,
3. split into training / validation / test subsets,
4. subtract the mean training image,
5. transpose images to channel-first format `(N, 3, 32, 32)`.

This keeps the later exercise cells close to the original notebook while matching the structure of this repo.

[2]:
```
cifar10_dir = repo_root / 'data' / 'cifar-10-batches-py'

if not cifar10_dir.exists():
    print('CIFAR-10 dataset not found. Downloading...')

    data_dir = repo_root / 'data'
    data_dir.mkdir(parents=True, exist_ok=True)

    url = 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz'
    archive_path = data_dir / 'cifar-10-python.tar.gz'

    urllib.request.urlretrieve(url, archive_path)

    print('Extracting dataset...')
    with tarfile.open(archive_path, 'r:gz') as tar:
        tar.extractall(path=data_dir)

    print('Download complete.')


def get_CIFAR10_data_local(
    num_training=49000,
    num_validation=1000,
    num_test=1000,
    subtract_mean=True,
):
    """Match the original assignment preprocessing using the local repo layout.
    """
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]

    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]

    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    if subtract_mean:
        mean_image = np.mean(X_train, axis=0)
        X_train = X_train - mean_image
        X_val = X_val - mean_image
        X_test = X_test - mean_image
```

```
    X_train = X_train.transpose(0, 3, 1, 2).copy()
    X_val = X_val.transpose(0, 3, 1, 2).copy()
    X_test = X_test.transpose(0, 3, 1, 2).copy()

    return {
        'X_train': X_train,
        'y_train': y_train,
        'X_val': X_val,
        'y_val': y_val,
        'X_test': X_test,
        'y_test': y_test,
    }


data = get_CIFAR10_data_local()
for k, v in list(data.items()):
    print(f'{k}: {v.shape}')
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.5 3. Initial loss and gradient check

Start by reading through `FullyConnectedNet` in `src/models/fc_net.py`.

At this stage, the core task is to implement: - parameter initialization, - the forward pass through an arbitrary number of hidden layers, - and the backward pass that produces gradients for all parameters.

The numeric gradient check below is the quickest sanity test for whether those pieces are wired correctly. As in the original assignment, relative errors around `1e-7` or smaller are a good sign.

[3]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print('Running check with reg = ', reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
```

```
        weight_scale=5e-2,
        dtype=np.float64,
    )

    loss, grads = model.loss(X, y)
    print('Initial loss: ', loss)

    # Most errors should be around e-7 or smaller.
    # It is acceptable to see W2 near e-5 when reg = 0.
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],␣
↪verbose=False, h=1e-5)
        print(f'{name} relative error: {rel_error(grad_num, grads[name])}')
```

```
Running check with reg =  0
Initial loss:  2.300479089768492
W1 relative error: 1.0252674442610356e-07
W2 relative error: 2.2120479308074287e-05
W3 relative error: 4.5623278305506856e-07
b1 relative error: 4.660094557753541e-09
b2 relative error: 2.085654124402131e-09
b3 relative error: 1.689724888469736e-10
Running check with reg =  3.14
Initial loss:  7.052114776533016
W1 relative error: 3.904542008453064e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989217647104e-08
b1 relative error: 1.4752429506250483e-08
b2 relative error: 1.4615869332918208e-09
b3 relative error: 1.3200479211447775e-10
```

1.6 4. Can the model overfit a tiny dataset?

A useful debugging trick is to deliberately overfit a very small training set. If optimization and backpropagation are implemented correctly, a sufficiently expressive network should drive the training accuracy to 100% on 50 examples.

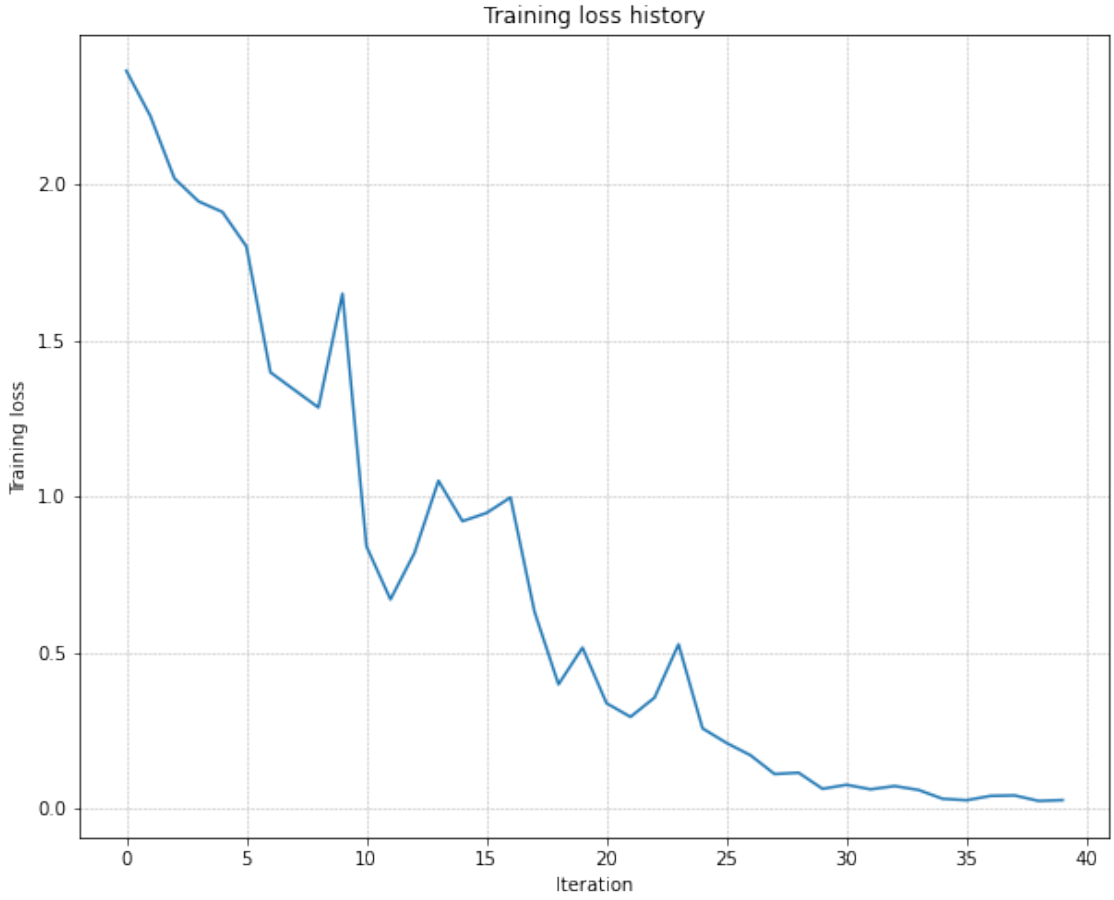
We first try a three-layer network, then a deeper five-layer one. The deeper model is usually more sensitive to learning rate and initialization scale, so this is also a practical test of optimization stability.

[4]:
```
# TODO: Use a three-layer net to overfit 50 training examples by
# tweaking the learning rate and initialization scale.

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
```

```
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

weight_scale = 1e-2   # Experiment with this.
learning_rate = 1e-2  # Experiment with this.

model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64,
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.363364
(Epoch 0 / 20) train acc: 0.180000; val_acc: 0.108000
(Epoch 1 / 20) train acc: 0.320000; val_acc: 0.127000
(Epoch 2 / 20) train acc: 0.440000; val_acc: 0.172000
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.184000
(Epoch 4 / 20) train acc: 0.540000; val_acc: 0.181000
(Epoch 5 / 20) train acc: 0.740000; val_acc: 0.190000
(Iteration 11 / 40) loss: 0.839976
(Epoch 6 / 20) train acc: 0.740000; val_acc: 0.187000
(Epoch 7 / 20) train acc: 0.740000; val_acc: 0.183000
(Epoch 8 / 20) train acc: 0.820000; val_acc: 0.177000
(Epoch 9 / 20) train acc: 0.860000; val_acc: 0.200000
(Epoch 10 / 20) train acc: 0.920000; val_acc: 0.191000
(Iteration 21 / 40) loss: 0.337174
(Epoch 11 / 20) train acc: 0.960000; val_acc: 0.189000
(Epoch 12 / 20) train acc: 0.940000; val_acc: 0.180000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.199000
```

```
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.199000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.195000
(Iteration 31 / 40) loss: 0.075911
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.182000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.201000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.207000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.185000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.192000
```



Now try a five-layer network with 100 hidden units in each hidden layer. You should still be able to overfit 50 examples within 20 epochs, but the tuning is usually less forgiving.

[5]:
```
num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}
```

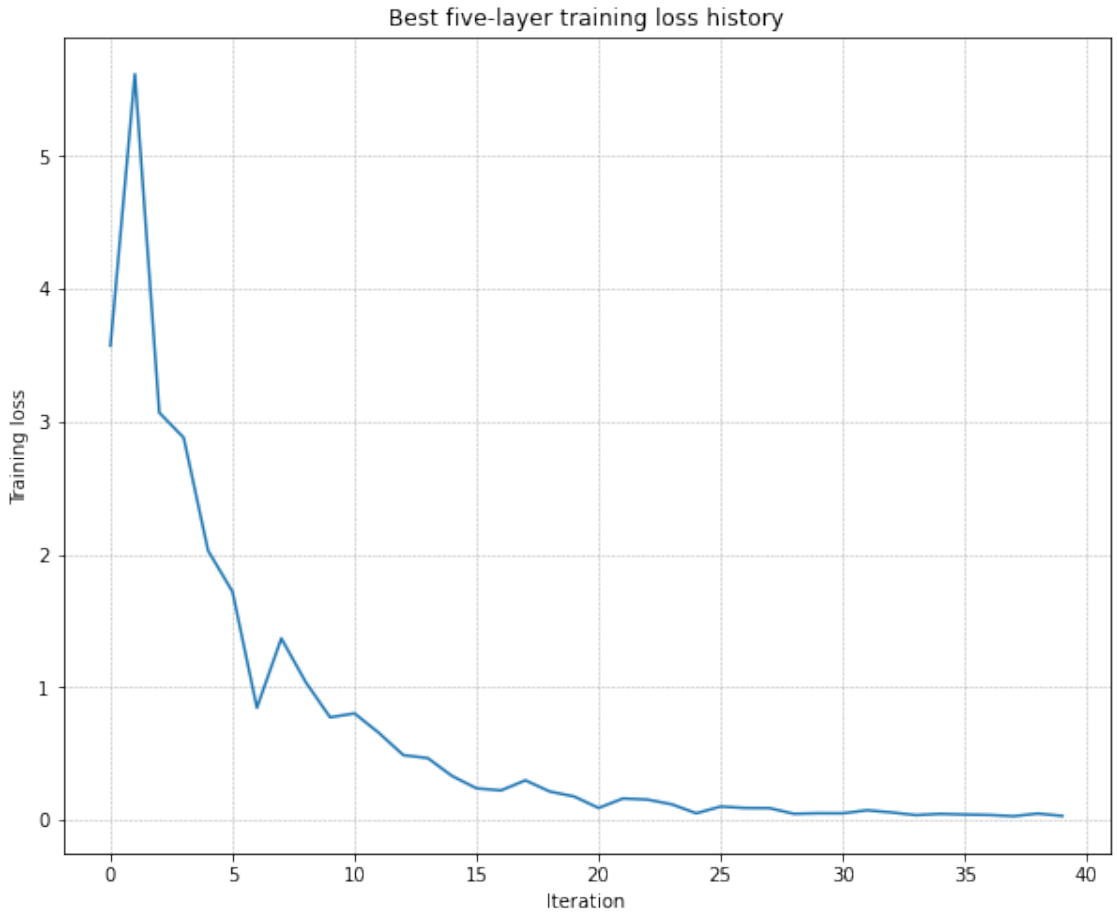
```
learning_rates = [1e-2, 5e-2, 1e-1, 2e-1]
weight_scales = [1e-3, 5e-3, 1e-2, 5e-2]

results = []
best_solver = None
best_model = None
best_train_acc = -1
best_lr = None
best_ws = None

for lr in learning_rates:
    for ws in weight_scales:
        model = FullyConnectedNet(
            [100, 100, 100, 100],
            weight_scale=ws,
            dtype=np.float64,
        )
        solver = Solver(
            model,
            small_data,
            print_every=10,
            num_epochs=20,
            batch_size=25,
            update_rule='sgd',
            optim_config={'learning_rate': lr},
            verbose=False,
        )
        solver.train()

        train_acc = solver.train_acc_history[-1]
        val_acc = solver.val_acc_history[-1]
        results.append((lr, ws, train_acc, val_acc))

        print(
            f"lr={lr:.1e}, ws={ws:.1e}, "
            f"train_acc={train_acc:.3f}, val_acc={val_acc:.3f}")
        if train_acc > best_train_acc:
            best_train_acc = train_acc
            best_model = model
            best_solver = solver
            best_lr = lr
            best_ws = ws
            best_model = model
            best_solver = solver
```

```
print(f"\nBest train acc: {best_train_acc:.3f} (lr={best_lr:.1e}, ws={best_ws:.
  ↪1e})")


plt.plot(best_solver.loss_history)
plt.title('Best five-layer training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
lr=1.0e-02, ws=1.0e-03, train_acc=0.160, val_acc=0.079
lr=1.0e-02, ws=5.0e-03, train_acc=0.160, val_acc=0.112
lr=1.0e-02, ws=1.0e-02, train_acc=0.160, val_acc=0.112
lr=1.0e-02, ws=5.0e-02, train_acc=1.000, val_acc=0.150
lr=5.0e-02, ws=1.0e-03, train_acc=0.160, val_acc=0.112
lr=5.0e-02, ws=5.0e-03, train_acc=0.160, val_acc=0.079
lr=5.0e-02, ws=1.0e-02, train_acc=0.180, val_acc=0.112
```

```
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/layers.py:886: RuntimeWarning:
divide by zero encountered in log
  loss = -np.sum(np.log(probs[np.arange(N), y])) / N
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/models/fc_net.py:348:
RuntimeWarning: overflow encountered in square
  loss += 0.5 * self.reg * np.sum(self.params["W%d" % i] ** 2)
/Users/cao20/opt/anaconda3/lib/python3.8/site-
packages/numpy/core/fromnumeric.py:86: RuntimeWarning: overflow encountered in
reduce
  return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/Users/cao20/cao20Lab/reborn/ml-from-scratch/src/models/fc_net.py:348:
RuntimeWarning: invalid value encountered in double_scalars
  loss += 0.5 * self.reg * np.sum(self.params["W%d" % i] ** 2)
```

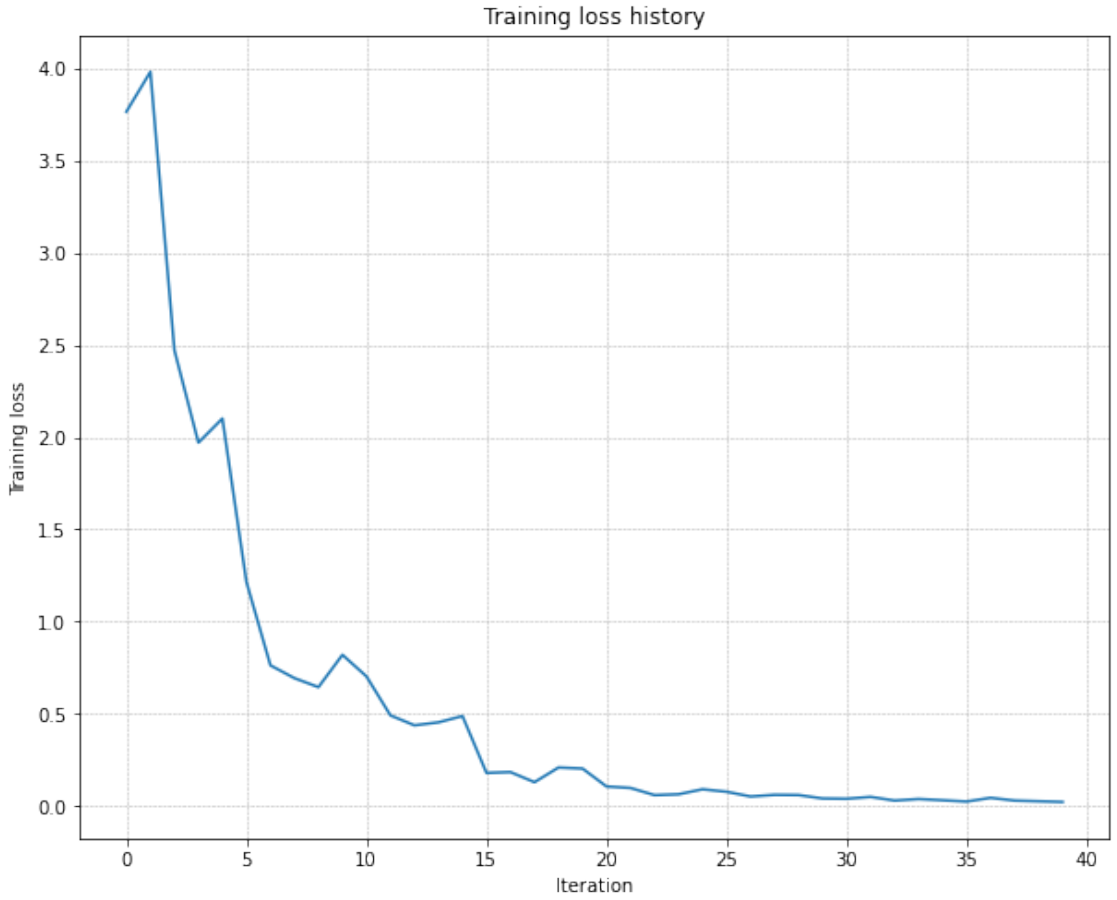
```
lr=5.0e-02, ws=5.0e-02, train_acc=0.080, val_acc=0.087
lr=1.0e-01, ws=1.0e-03, train_acc=0.160, val_acc=0.112
lr=1.0e-01, ws=5.0e-03, train_acc=0.160, val_acc=0.079
lr=1.0e-01, ws=1.0e-02, train_acc=0.220, val_acc=0.096
lr=1.0e-01, ws=5.0e-02, train_acc=0.080, val_acc=0.087
lr=2.0e-01, ws=1.0e-03, train_acc=0.160, val_acc=0.079
lr=2.0e-01, ws=5.0e-03, train_acc=0.160, val_acc=0.112
lr=2.0e-01, ws=1.0e-02, train_acc=0.200, val_acc=0.105
lr=2.0e-01, ws=5.0e-02, train_acc=0.080, val_acc=0.087

Best train acc: 1.000 (lr=1.0e-02, ws=5.0e-02)
```



[6]:
```
# TODO: Use a five-layer net to overfit 50 training examples by
# tweaking the learning rate and initialization scale.

num_train = 50
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

learning_rate = 1e-2  # Experiment with this.
weight_scale = 5e-2   # Experiment with this.

model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64,
```

```
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 3.764345
(Epoch 0 / 20) train acc: 0.300000; val_acc: 0.131000
(Epoch 1 / 20) train acc: 0.300000; val_acc: 0.110000
(Epoch 2 / 20) train acc: 0.420000; val_acc: 0.107000
(Epoch 3 / 20) train acc: 0.680000; val_acc: 0.139000
(Epoch 4 / 20) train acc: 0.800000; val_acc: 0.143000
(Epoch 5 / 20) train acc: 0.920000; val_acc: 0.166000
(Iteration 11 / 40) loss: 0.703532
(Epoch 6 / 20) train acc: 0.920000; val_acc: 0.144000
(Epoch 7 / 20) train acc: 0.980000; val_acc: 0.150000
(Epoch 8 / 20) train acc: 1.000000; val_acc: 0.140000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.149000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.160000
(Iteration 21 / 40) loss: 0.106949
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.149000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.158000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.151000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.157000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.161000
(Iteration 31 / 40) loss: 0.040295
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.165000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.167000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.165000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.166000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.163000
```



1.6.1 Inline Question 1

Did you notice anything about the comparative difficulty of training the three-layer network versus the five-layer network? Which network is more sensitive to the initialization scale, and why?

Answer.

The five-layer network is usually more sensitive. With more stacked affine and ReLU blocks, small changes in the initialization scale get amplified across depth: activations can shrink toward zero, blow up, or leave too many ReLUs inactive. That makes the deeper network harder to optimize with plain SGD, while the shallower three-layer network tends to tolerate a wider range of scales.

1.7 5. Update rules

So far we have used plain stochastic gradient descent. For deeper networks, optimization often improves significantly once we add momentum or adaptive per-parameter step sizes.

In this repo those update rules live in `src/optim.py`. The next sections mirror the original assignment: first verify each optimizer numerically, then compare training behavior on the same deep network.

1.8 6. SGD + Momentum

Implement `sgd_momentum` in `src/optim.py`, then run the reference check below. You should see errors around `1e-8` or smaller.

[7]:
```
from src.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-3, 'velocity': v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
    [0.1406, 0.20738947, 0.27417895, 0.34096842, 0.40775789],
    [0.47454737, 0.54133684, 0.60812632, 0.67491579, 0.74170526],
    [0.80849474, 0.87528421, 0.94207368, 1.00886316, 1.07565263],
    [1.14244211, 1.20923158, 1.27602105, 1.34281053, 1.4096],
])
expected_velocity = np.asarray([
    [0.5406, 0.55475789, 0.56891579, 0.58307368, 0.59723158],
    [0.61138947, 0.62554737, 0.63970526, 0.65386316, 0.66802105],
    [0.68217895, 0.69633684, 0.71049474, 0.72465263, 0.73881053],
    [0.75296842, 0.76712632, 0.78128421, 0.79544211, 0.8096],
])

print('next_w error: ', rel_error(next_w, expected_next_w))
print('velocity error: ', rel_error(expected_velocity, config['velocity']))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

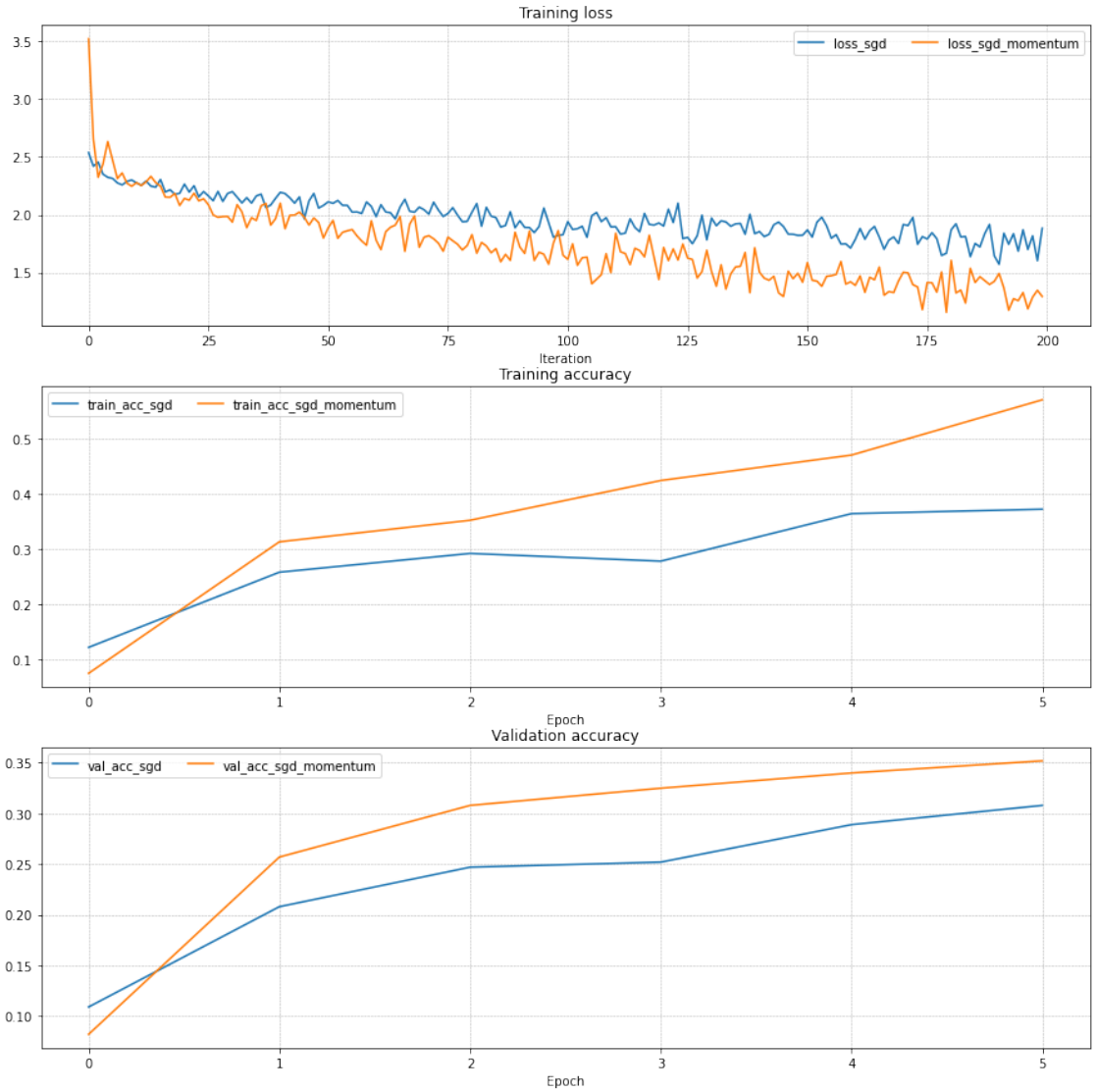
Once momentum is implemented, compare it against vanilla SGD on a six-layer network. The loss should usually fall faster with momentum.

[8]:
```
num_train = 4000
small_data = {
    'X_train': data['X_train'][:num_train],
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'],
    'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
```

```
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2,
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f'loss_{update_rule}')
    axes[1].plot(solver.train_acc_history, label=f'train_acc_{update_rule}')
    axes[2].plot(solver.val_acc_history, label=f'val_acc_{update_rule}')

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.537439
(Epoch 0 / 5) train acc: 0.122000; val_acc: 0.109000
(Iteration 11 / 200) loss: 2.275043
(Iteration 21 / 200) loss: 2.264691
(Iteration 31 / 200) loss: 2.199818
(Epoch 1 / 5) train acc: 0.258000; val_acc: 0.208000
(Iteration 41 / 200) loss: 2.194079
(Iteration 51 / 200) loss: 2.111654
(Iteration 61 / 200) loss: 1.983303
(Iteration 71 / 200) loss: 2.043094
```

```
(Epoch 2 / 5) train acc: 0.292000; val_acc: 0.247000
(Iteration 81 / 200) loss: 2.021877
(Iteration 91 / 200) loss: 1.947910
(Iteration 101 / 200) loss: 1.940256
(Iteration 111 / 200) loss: 1.895827
(Epoch 3 / 5) train acc: 0.278000; val_acc: 0.252000
(Iteration 121 / 200) loss: 1.901793
(Iteration 131 / 200) loss: 1.971557
(Iteration 141 / 200) loss: 1.853989
(Iteration 151 / 200) loss: 1.868130
(Epoch 4 / 5) train acc: 0.364000; val_acc: 0.289000
(Iteration 161 / 200) loss: 1.786761
(Iteration 171 / 200) loss: 1.915595
(Iteration 181 / 200) loss: 1.869732
(Iteration 191 / 200) loss: 1.571201
(Epoch 5 / 5) train acc: 0.372000; val_acc: 0.308000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 3.522364
(Epoch 0 / 5) train acc: 0.075000; val_acc: 0.082000
(Iteration 11 / 200) loss: 2.277267
(Iteration 21 / 200) loss: 2.141035
(Iteration 31 / 200) loss: 1.936170
(Epoch 1 / 5) train acc: 0.313000; val_acc: 0.257000
(Iteration 41 / 200) loss: 2.099330
(Iteration 51 / 200) loss: 1.886719
(Iteration 61 / 200) loss: 1.779531
(Iteration 71 / 200) loss: 1.805186
(Epoch 2 / 5) train acc: 0.352000; val_acc: 0.308000
(Iteration 81 / 200) loss: 1.827750
(Iteration 91 / 200) loss: 1.719926
(Iteration 101 / 200) loss: 1.613538
(Iteration 111 / 200) loss: 1.840272
(Epoch 3 / 5) train acc: 0.424000; val_acc: 0.325000
(Iteration 121 / 200) loss: 1.716426
(Iteration 131 / 200) loss: 1.508034
(Iteration 141 / 200) loss: 1.502036
(Iteration 151 / 200) loss: 1.585279
(Epoch 4 / 5) train acc: 0.470000; val_acc: 0.340000
(Iteration 161 / 200) loss: 1.388295
(Iteration 171 / 200) loss: 1.501785
(Iteration 181 / 200) loss: 1.604943
(Iteration 191 / 200) loss: 1.491604
(Epoch 5 / 5) train acc: 0.570000; val_acc: 0.352000
```



1.9 7. RMSProp and Adam

Implement `rmsprop` and `adam` in `src/optim.py`, then verify them with the reference checks below. As in the original CS231n notebook, use the full Adam update with bias correction.

[9]:
```
from src.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
```

```
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
    [-0.132737, -0.08078555, -0.02881884, 0.02316247, 0.07515774],
    [0.12716641, 0.17918792, 0.23122175, 0.28326742, 0.33532447],
    [0.38739248, 0.43947102, 0.49155973, 0.54365823, 0.59576619],
])
expected_cache = np.asarray([
    [0.5976, 0.6126277, 0.6277108, 0.64284931, 0.65804321],
    [0.67329252, 0.68859723, 0.70395734, 0.71937285, 0.73484377],
    [0.75037008, 0.7659518, 0.78158892, 0.79728144, 0.81302936],
    [0.82883269, 0.84469141, 0.86060554, 0.87657507, 0.8926],
])

print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

[10]:
```
from src.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N * D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N * D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N * D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N * D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
    [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
    [-0.1380274, -0.08544591, -0.03286534, 0.01971428, 0.0722929],
    [0.1248705, 0.17744702, 0.23002243, 0.28259667, 0.33516969],
    [0.38774145, 0.44031188, 0.49288093, 0.54544852, 0.59801459],
])
expected_v = np.asarray([
    [0.69966, 0.68908382, 0.67851319, 0.66794809, 0.65738853],
    [0.64683452, 0.63628604, 0.6257431, 0.61520571, 0.60467385],
    [0.59414753, 0.58362676, 0.57311152, 0.56260183, 0.55209767],
    [0.54159906, 0.53110598, 0.52061845, 0.51013645, 0.49966],
])
expected_m = np.asarray([
    [0.48, 0.49947368, 0.51894737, 0.53842105, 0.55789474],
    [0.57736842, 0.59684211, 0.61631579, 0.63578947, 0.65526316],
```

```
    [0.67473684, 0.69421053, 0.71368421, 0.73315789, 0.75263158],
    [0.77210526, 0.79157895, 0.81105263, 0.83052632, 0.85],
])

print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

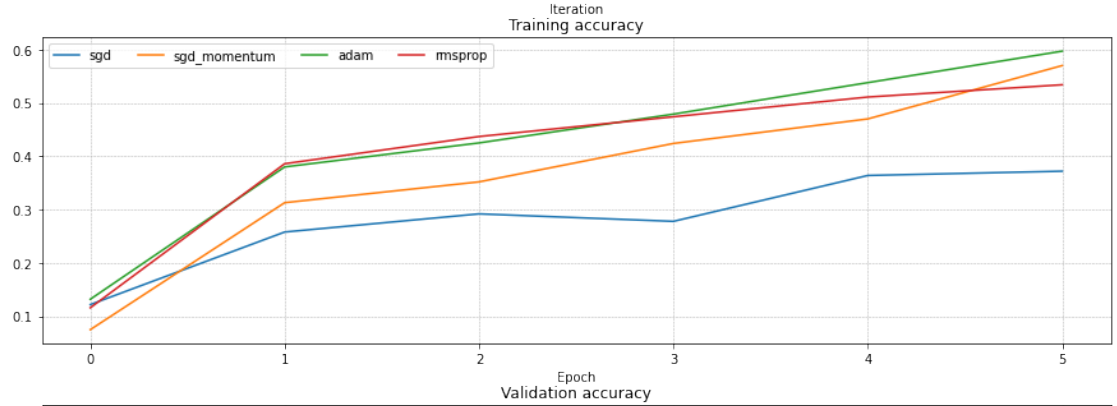
Once those optimizers are working, compare deep-network training with Adam and RMSProp. Adaptive methods typically converge faster than plain SGD on this task.

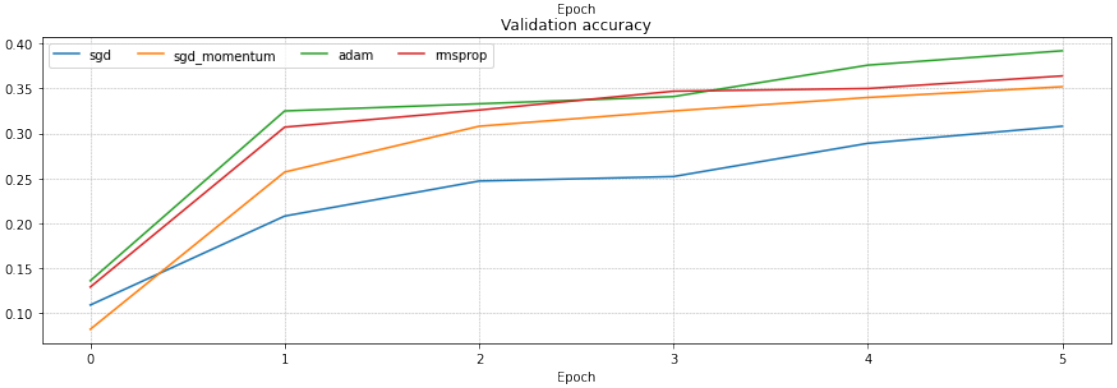
[11]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2,
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=update_rule)
    axes[1].plot(solver.train_acc_history, label=update_rule)
    axes[2].plot(solver.val_acc_history, label=update_rule)
```

```python
for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 2.510546
(Epoch 0 / 5) train acc: 0.132000; val_acc: 0.136000
(Iteration 11 / 200) loss: 2.195202
(Iteration 21 / 200) loss: 2.078113
(Iteration 31 / 200) loss: 1.812428
(Epoch 1 / 5) train acc: 0.380000; val_acc: 0.325000
(Iteration 41 / 200) loss: 1.826273
(Iteration 51 / 200) loss: 1.863777
(Iteration 61 / 200) loss: 1.567748
(Iteration 71 / 200) loss: 1.867368
(Epoch 2 / 5) train acc: 0.425000; val_acc: 0.333000
(Iteration 81 / 200) loss: 1.527325
(Iteration 91 / 200) loss: 1.566275
(Iteration 101 / 200) loss: 1.528926
(Iteration 111 / 200) loss: 1.473175
(Epoch 3 / 5) train acc: 0.479000; val_acc: 0.341000
(Iteration 121 / 200) loss: 1.449963
(Iteration 131 / 200) loss: 1.246596
(Iteration 141 / 200) loss: 1.442075
(Iteration 151 / 200) loss: 1.444211
(Epoch 4 / 5) train acc: 0.538000; val_acc: 0.376000
(Iteration 161 / 200) loss: 1.385134
(Iteration 171 / 200) loss: 1.242322
(Iteration 181 / 200) loss: 1.218187
(Iteration 191 / 200) loss: 1.452783
(Epoch 5 / 5) train acc: 0.597000; val_acc: 0.392000

Running with  rmsprop
(Iteration 1 / 200) loss: 2.913589
(Epoch 0 / 5) train acc: 0.116000; val_acc: 0.129000
(Iteration 11 / 200) loss: 2.127827
(Iteration 21 / 200) loss: 1.958286
(Iteration 31 / 200) loss: 1.864374
(Epoch 1 / 5) train acc: 0.386000; val_acc: 0.307000
(Iteration 41 / 200) loss: 1.856374
(Iteration 51 / 200) loss: 1.676541
(Iteration 61 / 200) loss: 1.862084
(Iteration 71 / 200) loss: 1.753047
(Epoch 2 / 5) train acc: 0.437000; val_acc: 0.326000
(Iteration 81 / 200) loss: 1.675834
```

```
(Iteration 91 / 200) loss: 1.772902
(Iteration 101 / 200) loss: 1.511201
(Iteration 111 / 200) loss: 1.548312
(Epoch 3 / 5) train acc: 0.474000; val_acc: 0.347000
(Iteration 121 / 200) loss: 1.477052
(Iteration 131 / 200) loss: 1.609006
(Iteration 141 / 200) loss: 1.553027
(Iteration 151 / 200) loss: 1.435824
(Epoch 4 / 5) train acc: 0.511000; val_acc: 0.350000
(Iteration 161 / 200) loss: 1.580685
(Iteration 171 / 200) loss: 1.386873
(Iteration 181 / 200) loss: 1.506295
(Iteration 191 / 200) loss: 1.394509
(Epoch 5 / 5) train acc: 0.534000; val_acc: 0.364000
```





1.9.1 Inline Question 2

AdaGrad uses the update rule

```
cache += dw ** 2
w += -learning_rate * dw / (np.sqrt(cache) + eps)
```

Why can AdaGrad's updates become very small over time? Would Adam have the same issue?

Answer.

AdaGrad keeps adding squared gradients into `cache`, so the denominator grows monotonically. Even if the current gradient stays informative, the effective step size keeps shrinking because it is divided by a larger and larger accumulated history. That can make learning stall.

Adam is much less prone to that exact failure mode because it uses **exponentially decaying moving averages** of both the gradient and the squared gradient. In other words, Adam forgets old information instead of accumulating it forever, so the denominator does not grow without bound in the same way.

1.10 8. Train a strong fully connected model

Now train the best fully connected model you can on CIFAR-10 and store it in `best_model`. A reasonable target is **at least 50% validation accuracy**; with careful tuning, higher results are possible.

This repo version keeps the search intentionally small so it stays within roughly **5 minutes** on a typical local setup. Expand the grid later if you want a stronger model.

[15]:
```
best_model = None
best_val_acc = -1.0
best_stats = None

configs = [
    {
        'hidden_dims': [256, 256],
        'learning_rate': 1e-3,
        'weight_scale': 2e-2,
        'reg': 1e-3,
        'normalization': 'batchnorm',
        'update_rule': 'adam',
        'num_epochs': 10,
        'batch_size': 200,
    }
]

for cfg in configs:
    model = FullyConnectedNet(
```

```
        cfg['hidden_dims'],
        reg=cfg['reg'],
        weight_scale=cfg['weight_scale'],
        normalization=cfg['normalization'],
    )

    solver = Solver(
        model,
        data,
        update_rule=cfg['update_rule'],
        optim_config={'learning_rate': cfg['learning_rate']},
        lr_decay=0.95,
        num_epochs=cfg['num_epochs'],
        batch_size=cfg['batch_size'],
        print_every=100,
        verbose=True,
    )

    solver.train()

    train_acc = solver.train_acc_history[-1]
    val_acc = solver.best_val_acc

    print(
        f"hidden_dims={cfg['hidden_dims']}, "
        f"lr={cfg['learning_rate']}, "
        f"weight_scale={cfg['weight_scale']}, "
        f"reg={cfg['reg']}, "
        f"normalization={cfg['normalization']}, "
        f"update_rule={cfg['update_rule']}, "
        f"train_acc={train_acc:.4f}, val_acc={val_acc:.4f}"
    )

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model = model
        best_stats = {
            **cfg,
            'train_acc': train_acc,
            'val_acc': val_acc,
        }

print('best_val_acc:', best_val_acc)
print('best_stats:', best_stats)
```

```
(Iteration 1 / 2450) loss: 2.514464
(Epoch 0 / 10) train acc: 0.255000; val_acc: 0.244000
(Iteration 101 / 2450) loss: 1.649229
```

```
(Iteration 201 / 2450) loss: 1.525518
(Epoch 1 / 10) train acc: 0.499000; val_acc: 0.471000
(Iteration 301 / 2450) loss: 1.585913
(Iteration 401 / 2450) loss: 1.464557
(Epoch 2 / 10) train acc: 0.539000; val_acc: 0.512000
(Iteration 501 / 2450) loss: 1.522722
(Iteration 601 / 2450) loss: 1.383585
(Iteration 701 / 2450) loss: 1.269106
(Epoch 3 / 10) train acc: 0.565000; val_acc: 0.509000
(Iteration 801 / 2450) loss: 1.485525
(Iteration 901 / 2450) loss: 1.340550
(Epoch 4 / 10) train acc: 0.594000; val_acc: 0.540000
(Iteration 1001 / 2450) loss: 1.461776
(Iteration 1101 / 2450) loss: 1.340278
(Iteration 1201 / 2450) loss: 1.360117
(Epoch 5 / 10) train acc: 0.590000; val_acc: 0.536000
(Iteration 1301 / 2450) loss: 1.327544
(Iteration 1401 / 2450) loss: 1.194441
(Epoch 6 / 10) train acc: 0.618000; val_acc: 0.525000
(Iteration 1501 / 2450) loss: 1.273470
(Iteration 1601 / 2450) loss: 1.169227
(Iteration 1701 / 2450) loss: 1.178053
(Epoch 7 / 10) train acc: 0.610000; val_acc: 0.539000
(Iteration 1801 / 2450) loss: 1.197181
(Iteration 1901 / 2450) loss: 1.186229
(Epoch 8 / 10) train acc: 0.651000; val_acc: 0.563000
(Iteration 2001 / 2450) loss: 1.179422
(Iteration 2101 / 2450) loss: 1.284623
(Iteration 2201 / 2450) loss: 1.161779
(Epoch 9 / 10) train acc: 0.672000; val_acc: 0.551000
(Iteration 2301 / 2450) loss: 1.066729
(Iteration 2401 / 2450) loss: 0.995506
(Epoch 10 / 10) train acc: 0.701000; val_acc: 0.551000
hidden_dims=[256, 256], lr=0.001, weight_scale=0.02, reg=0.001,
normalization=batchnorm, update_rule=adam, train_acc=0.7010, val_acc=0.5630
best_val_acc: 0.563
best_stats: {'hidden_dims': [256, 256], 'learning_rate': 0.001, 'weight_scale':
0.02, 'reg': 0.001, 'normalization': 'batchnorm', 'update_rule': 'adam',
'num_epochs': 10, 'batch_size': 200, 'train_acc': 0.701, 'val_acc': 0.563}
```

1.11 9. Final evaluation

Run the best model on the validation and test sets. As in the earlier notebooks, saving the final model makes it easy to reuse later without retraining.

[16]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
```

```
print('Validation set accuracy: ', np.mean(y_val_pred == data['y_val']))
print('Test set accuracy: ', np.mean(y_test_pred == data['y_test']))
```

```
Validation set accuracy:  0.549
Test set accuracy:  0.543
```

[17]:
```
# Save best model
best_model.save('best_fully_connected_net.npy')
```

```
best_fully_connected_net.npy saved.
```

1.12 10. Takeaways

This notebook extends the earlier two-layer network exercise in two important ways:

- **depth** gives the model more expressive power, but also makes optimization more fragile;
- **optimizer choice** matters much more once the network gets deeper;
- **small-set overfitting tests** are one of the fastest ways to debug an implementation;
- **gradient checks** are still the most reliable correctness check before large training runs.

That combination of modular implementation and careful optimization is the main lesson of the fully connected network assignment.